Kurdistan Regional Government – Iraq
Ministry of Higher Education and Scientific Research
University of Duhok
College of Science
Department of Computer Science

Evaluating Large Language Models (LLMs) for Static Code Analysis

A Thesis

Submitted to the Council of the College of Science/University of Duhok in Partial Fulfilment of the Requirements for the Degree of Master of Science (M.Sc.) in Computer Science

By

Hekar Azwar Mohammed Salih

B.Sc. in Computer Science, University of Nawroz – 2014

Supervised by

Assistant Professor Dr. Qusay Idrees Sarhan

University of Duhok, Duhok, Kurdistan Region, Iraq

1447 A.H. 2026 A.D. 2726 K.

Supervisor Certification

I certify that this thesis, entitled **"Evaluating Large Language Models (LLMs) for Static Code Analysis",** is prepared by "**Hekar Azwar Mohammed Salih**" under my supervision at the Department of Computer Science, College of Science - University of Duhok, as a partial fulfillment of the requirements for the degree of Master of Science (MSc) in Computer Science.

Signature:

Supervisor: Assistant Professor Dr. Qusay Idrees Sarhan

Department of Computer Science

College of Science

University of Duhok

Date: / /2026

Declaration and Recommendation of Linguistic Evaluator

I hereby declare that this thesis, entitled **"Evaluating Large Language Models (LLMs) for Static Code Analysis"**, has undergone linguistic review and that all linguistic and expressive errors have been fixed. As a result, the thesis is now eligible for discussion in terms of linguistic integrity and accurate expression.

Signature:

Assistant Professor Dr. Lolav Mohammed Hassan Mohammed Salih Alhamid
Department of English Language and Literature
College of Languages
University of Duhok

Date: / /2026

Recommendation of the Head of the Academic Department

Based on the recommendations submitted by the supervisor and the linguistic and scientific evaluator, I nominate this thesis for discussion.

Signature:

Assistant Professor Dr. Dezheen Hussein Abdulazeez

Department of Computer Science

College of Science

University of Duhok

Date: / /2026

Examination Committee Certification

We hereby certify that we have reviewed the thesis entitled "**Evaluating Large Language Models (LLMs) for Static Code Analysis**" and have examined the candidate, **Hekar Azwar Mohammed Salih**, as the Examining Committee, we find that the thesis is satisfactory and adequate for the award of the degree of Master of Science (MSc) in Computer Science.

Signature:
Committee Chairman: Professor Dr. Adel Sabry Eesa
Date: / /2026

Signature:
Committee Member: Assistant Professor Dr. Ahmad Baheej Yassin Al-Khalil
Date: / /2026

Signature:
Committee Member: Assistant Professor Dr. Naaman Omar Yaseen
Date: / /2026

Signature:
Committee Member (Supervisor): Assistant Professor Dr. Qusay Idrees Sarhan
Date: / /2026

Approval of the Dean of College of Science / University of Duhok

Signature:

Name: Assistant Professor Dr. Mohammed Aziz Ibrahem
Date: / /2026

Declaration

I hereby declare that the work described in this thesis is original work prepared and written by me for the degree of Master of Science (MSc) in Computer Science, at the Department of Computer Science, College of Science, University of Duhok. Additionally, the results found are entirely my own work, and I have faithfully and properly cited all the sources used in the thesis.

Signature:

Hekar Azwar Mohammed Salih
Date: / /2026

Acknowledgements

I would like to thank my supervisor (Dr. Qusay Idrees Sarhan) for his guidance, support and effective feedback during the period of my master study. This thesis would not have been completed without his expertise and motivation at every step of the way. Special thanks to the University of Duhok, College of Science and Department of Computer Science, for providing an ideal environment and facilities that without which the execution of this thesis would have been impossible.

I would also like to thank my colleagues and supporting staff who aided me both technically and administratively, especially for those who helped me with proofreading, editing, etc. Thanks also to my beloved wife, who stood by my side and encouraged me through all the stressful times of this journey due to her patience, understanding and professional and emotional support, which made me concentrated enough to complete this thesis. Last but not least, I would like to extend my sincere thanks to my family and friends who stood by my side, supported me, motivated me and always believed in me during such a professional undertaking. Without their encouragement, I would have lost my concentration many times.

Hekar Azwar Mohammed Salih

June 2026

Dedication

This thesis is affectionately dedicated to my beloved wife, for her support, patience and encouragement have been tremendous throughout this journey. I also dedicate this achievement to my family, who have always supported me with steadfast faith and love, and to all who have inspired, helped and journeyed with me to this point; this achievement is for you.

Hekar Azwar Mohammed Salih

June 2026

List of Publications

This thesis has resulted in the following peer-reviewed publications:

1. H. A. Mohammed Salih and Q. I. Sarhan, **"A Systematic Survey on Large Language Models for Static Code Analysis"**, *ARO,* vol. 13, no. 1, pp. 251–265, Jun. 2025. https://doi.org/10.14500/aro.12082
   Impact Factor: 2.1 (as of 2025).

2. H. A. Mohammed Salih and Q. I. Sarhan, **"A Study of Large Language Models in Detecting Python Code Violations"**, *ARO,* vol. 13, no. 2, pp. 215–225, Oct. 2025. https://doi.org/10.14500/aro.12395
   Impact Factor: 2.1 (as of 2025).

Abstract

Following good coding practices significantly enhances the readability, maintainability, and reliability of software. Static code analysis tools commonly used for Python code, like Pylint and Flake8, evaluate Python code for quality without executing it. However, they have limited capabilities to offer a good coding understanding and reasoning within context. This thesis investigates how well Large Language Models (LLMs) compare to static code analysis tools in detecting coding violations in Python, measuring the performance of six state-of-the-art LLMs: ChatGPT, Gemini, Claude Sonnet, DeepSeek, Kimi, and Qwen against Pylint and Flake8, using a curated dataset of 75 Python code segments tagged with 27 common coding violations. In addition, three common prompting techniques, structural prompting, chain-of-thought prompting, and role-based prompting, were used to prompt the selected LLMs, analyzing the impact of these techniques on the performance of selected LLMs.

Experimental results reveal that the best performer, Claude Sonnet, scored the highest F1-Score of (0.81), surpassing Flake8 with an F1-Score of (0.79) and Gemini with an F1-Score of (0.78). Claude's Precision (0.99) and Recall (0.69) scores were also high but the other LLMs achieved mixed results, where Kimi, DeepSeek, Qwen, and ChatGPT all fell short of others. Pylint and ChatGPT showed the weakest performance with an F1-Score of (0.35) and (0.13) respectively, owing primarily to extremely low Recall scores of (0.21) and (0.07). Analysis of the results also reveals that there is high variance in reliability, where Kimi has perfect False Discovery Rate (0.00), while Qwen has extreme variability in its measurements with a False Discovery Rate of up to (0.92) in certain prompting conditions. Qualitatively, the models exhibited impressive capabilities in detecting documentation and stylistic violations that Pylint and Flake8 could not detect but also failed to identify complex structural violations even when they achieved near-perfect Precision scores. The results were also influenced by the prompting technique employed, where structural prompting was the most balanced technique in most results. Furthermore, the study also shows that the input strategy is a useful variable, where treating code segments individually as input for the models rather than in batches yielded universal performance increases, achieving the maximum of (+0.64) F1-Score improvement for ChatGPT as most notably improving. This thesis provides empirical evidence on using LLMs for code quality and demonstrates their potential as complementary static code analysis tools for Python.

Keywords:

Large Language Models, Static Code Analysis, Code Quality, Code Violation, Python.

Table of Contents

List of Figures

List of Tables

List of Abbreviations

Abbreviation	Definition
AI	Artificial Intelligence
API	Application Programming Interface
CG	Call Graph
CoT	Chain of Thought
CR	Consistency Rate
CVE	Common Vulnerabilities and Exposures
FDR	False Discovery Rate
FN	False Negative
FP	False Positive
IoT	Internet of Things
LLMs	Large Language Models
NLP	Natural Language Processing
PA	Prompt Agreement
PEP-8	Python Enhancement Proposal 8
PMD	Programming Mistake Detector
RQs	Research Questions
SDLC	Software Development Life Cycle
TP	True Positive

Chapter One
Introduction

Chapter 1: Introduction

1.1 Overview and Background

Adherence to good coding in modern software engineering continues to be relevant to the reliability, maintainability, and security of any software product [1], [2]. One type of poor coding, termed coding violations, is related to the so-called "code smells" which can reduce quality dramatically and lead to costly issues that are expensive to fix later in the Software Development Life Cycle (SDLC) [3]. Aiming to reduce poor coding, static code analysis tools are emerging as crucial additions to the quality assurance processes in the development of quality software products [4]. Static code analysis tools such as Pylint, Flake8, PMD, FindBugs and SonarQube have been developed for codes written using different programming languages, these tools work by analyzing source code without executing it, and can identify violations that include coding defects, vulnerabilities, and stylistic issues [5]. Although such tools can produce false positives that can be overwhelming for a developer to check, the usage of static code analysis tools can identify early coding issues in software products, minimizing maintenance costs later in the SDLC [6].

In recent years, Large Language Models (LLMs) like ChatGPT are having a huge impact on software engineering and are revolutionizing many aspects of the field [7], [8]. LLMs are trained on different datasets that include natural language texts as well as source codes. As a consequence, they show astonishing capabilities of natural language understanding as well as source code understanding. LLMs can therefore perform code generation, code review, and code repair tasks. Their strength lies in their capability for code understanding and detecting code patterns related to violations. They also provide help with automated code improvements using natural language commands [9], [10]. However, understanding code is often difficult for LLMs, since LLMs are prone to hallucinations that can lead to perform incorrect analysis of codes with quality issues [11]–[13]. The aim of this thesis is to evaluate different LLMs for detecting Python coding violations. Python was selected over other programming languages for many reasons such as it is currently ranked as the number one programming language in 2026 according to the TIOBE Index [1]. Furthermore, it is the most widely used language in Artificial Intelligence (AI) development due to its simple syntax, extensive libraries (such as TensorFlow, PyTorch, and scikit-learn), and strong community support [14]. Moreover, Python has become a dominant language for the Internet of Things (IoT), where it is used for programming IoT constrained devices (such as Raspberry Pi and Pyboard), and building rapid

[1] https://www.tiobe.com/tiobe-index/

IoT systems [15]. Its high-level abstractions and many IoT libraries (e.g., paho-mqtt and adafruit-io) make Python especially suitable for connecting devices, managing data, and enabling communication in IoT environments. As IoT increasingly integrate intelligence capabilities, the ties between Python, AI, and IoT enable the development of innovation systems, making the application of static code analysis becomes even more important.

In the thesis, various LLMs are assessed for their applicability to performing static code analysis for Python codes. Therefore, an introduction, in the following subsections, is needed to static code analysis and to LLMs for static code analysis.

1.1.1 Static Code Analysis

Static code analysis is the evaluation of code that does not require execution opposed to dynamic code analysis which requires code execution and it is used to assess code quality in all types of software and hardware systems [16]. Static code analysis tools search for bugs and security vulnerabilities, and check adherence to coding standards [17]. They scan code files to find issues using a pattern-matching, rule-based approach. More advanced static code analysis tools use data-flow analysis to find out what values can be assigned to variables and how untrusted data can flow through the application. This is essential to detect security issues like SQL injection or buffer overflow [18]. The following Python code example has a number of flaws that Python static code analysis tools can detect.

```
def calculate_average(numbers):
    total = 0
    count = 0
    for i in range(len(numbers)):
        total = total + numbers[i]
        count = count + 1
    return total / count
```

Listed below are the issues that can be detected:

- **Lack of Input Validation**
  - No check to ensure *numbers* is not *None* or contains only numeric values.
- **Inefficient Looping**
  - Using *range(len(numbers))* instead of iterating directly over the list.
- **Redundant Variable**
  - count is unnecessary because *len(numbers)* can be used directly.
- **Potential Division by Zero**
  - Division by zero can happen with *total / count*.
- **Maintainability Issue**
  - Function can be simplified using built-in functions like *sum()*.

1.1.2 LLMs for Static Code Analysis

LLMs represent a significant advancement in the field of Natural Language Processing (NLP) and AI. They are large deep neural network-based pre-trained language models, typically with billions of parameters, that understand commands in natural languages and generates responses/answers in multiple forms. The transformer architecture, which is based on the self-attention mechanism to capture long-range dependencies, is the state-of-the-art architecture for modern LLMs. [19]. They have been used in different SDLC activities, including requirement engineering, code generation, vulnerability detection, and even in software testing [20] as they have emergent abilities, like in-context learning and reasoning [21].

LLMs act as static code analyzers, leveraging their training across extensive codebases to identify bugs, security vulnerabilities, and quality concerns without executing code [22], [23]. Unlike the static code analysis tools currently available, which rely on predefined rules, LLMs can be considered general code analyzers that can identify logic bugs related to the misuse of variables that standard static analyzers would overlook [24], [25]. Beyond the filtering of false positive warnings issued by well-established static analyzers to alleviate the burden placed on the developer by such warnings [5]. LLMs enhance the response with a detailed human-readable overview of the identified issues, and by automatically generating an appropriate fix to the identified issues, effectively taking on the role of an interactive coding assistant [26].

The following example demonstrates how the developer can prompt an LLM such as ChatGPT to conduct a static code analysis on a specified code segment:

"Check the following Python code for PEP 8 violations or other issues and return a table with the following columns: Severity (Warning, Error, etc.), Message (Descriptive Message), Type (Type of issue), and Line Number (Line where the issue starts)".

```
def addDigits(num):
    while num>=10:
        num = sum(map(int, str(num)))
    return num
```

The LLMs generated output should be near to the format presented in Table 1.1:

Table 1.1: LLM-identified coding issues

Severity	Message	Type	Line Number
Warning	Function name should be lowercase with words separated by underscores (add_digits).	Function naming conventions	1
Warning	Missing spaces around comparison operator (>=).	Whitespace in expressions	2

1.2 Problem Statement

Although the popularity of Python and the importance of adhering to good coding practices, there is, however, limited research that systematically analyzes Python code violations and code quality using LLMs. Hence, the degree to which LLMs can detect such violations remains unknown. This thesis attempts to address the shortcoming through assessing the strengths and weaknesses of multiple LLMs for detecting Python coding violations, evaluating the performance of LLMs against traditional static code analysis tools, and investigating the effect of prompting strategies on their performance.

1.3 Aim and Objectives

The aim of this thesis is to investigate the capabilities and limitations of various LLMs in detecting Python coding violations depending on Python Enhancement Proposal 8 (PEP-8)[2] and violations of coding best practices.

In addition, the specific objectives of this thesis are:

1. To evaluate the effectiveness of LLMs in detecting Python coding violations against traditional Python static code analysis tools.
2. To provide a taxonomy that classifies Python coding violations and highlights where LLMs perform better or worse.
3. To analyze how consistent the selected LLMs are in identifying the same coding violations.
4. To investigate the impact of granularity by comparing LLMs' performance on batch code segments versus individual code segments.
5. To build a specialized dataset for evaluating Python coding violations.

[2] https://peps.python.org/pep-0008/

1.4 Contributions

This thesis achieves several contributions as follows:

1. Comparative Effectiveness Analysis: Evaluates how well LLMs detect Python coding violations against traditional Python static code analysis tools.
2. Taxonomy of Violations: Provides a taxonomy that classifies Python coding violations, showing where LLMs perform better or worse, thereby highlighting model strengths and limitations.
3. Cross-Model Consistency: Analyzes six state-of-the-art LLMs to identify consistent model-specific patterns across Python violation detection.
4. Batch vs. Individual Code Analysis: Investigates the impact of granularity by comparing LLMs performance on Python batch code segments versus individual segments analysis.
5. Dataset Construction: Builds and releases a new, specialized dataset for evaluating Python coding violations, providing a standard for benchmarking current and future LLMs.

1.5 Limitations

While this thesis provides experimental findings about LLMs for static code analysis, the following limitations should be acknowledged:

- It is limited to a small set of PEP-8 and best practices coding violations of Python. Python is a widely used programming language, but its findings cannot be directly generalized to other programming languages with other syntactic structures, design paradigms, and coding standards.
- The used dataset consists of 75 Python code segments. The dataset was carefully selected and manually annotated with 27 common coding violations. However, it is limited in scope and does not replicate the scale of large, real-world software systems comprising multiple files, modules, and internal and external dependencies.
- The comparison with traditional static code analysis tools is limited to Pylint and Flake8. While these tools are widely adopted and appropriate for Python code quality assessment, other existing tools were not included.

- It examines only three prompting strategies: structural, chain-of-thought, and role-based prompting. Other prompting techniques, such as few-shot prompting, retrieval-augmented prompting, or tool-assisted prompting, were not explored and may further influence detection performance.
- The selected LLMs were evaluated as tools only using publicly available user interfaces without fine-tuning or internal configuration control. As a result, the experiments were conducted in a black-box setting, and the non-deterministic nature of LLM outputs may impact result reproducibility.
- The non-deterministic nature of LLM outputs, the same input may produce different responses across multiple executions, can affect the consistency and reproducibility of the results.

1.6 Thesis Outline

This thesis is divided into several chapters, as follows:

- **Chapter One (Introduction)**, which is the current chapter, introduces the background and motivation of the study, highlighting the importance of good coding practices, static code analysis and the emerging importance of LLMs in the domain of software engineering. It presents the problem statement, aims and objectives, thesis contributions and limitations.
- **Chapter Two (Literature Survey)** presents a systematic literature survey on the topic of this thesis, covering LLMs for static code analysis and also discusses existing applications, datasets, tools, and limitations related to LLM-based static code analysis.
- **Chapter Three (Research Methodology)** describes the research methodology, dataset, and LLMs employed in the experimental part of this thesis. It outlines the overall research methodology of the study, including the research questions, the dataset construction and annotation, as well as, the selection of the LLMs and static code analysis tools, the prompt engineering that was performed, and the evaluation methods that were used for evaluating the ability of the LLMs to detect Python coding violations.
- **Chapter Four (Results and Discussion)** presents the experimental results of this thesis and their discussion. It addresses each research question of this thesis. Also, it highlights the benefits and limitations of using LLMs for Python static code analysis.
- **Chapter Five (Conclusions and Future Works)** presents the main findings and conclusions of this thesis and provides suggestions for future works.

Chapter Two
Literature Survey

Chapter 2: Literature Survey

2.1 Introduction

Static code analysis is a form of code analysis that helps to find issues in a code without executing it [18]. Traditional static code analysis tools are usually rule-based and heuristic-defined and they have weak contextual reasoning, especially to the types of modern complex systems and coding styles that exist today. Static code analysis is essential in software engineering, cybersecurity, and the IoT. For example, in software engineering, static code analysis can be used to check code before actual deployment. The result is that competent, high-quality, reliable, and error-free software programs can be created [27]. In cybersecurity, static code analysis can identify and help locate and understand vulnerabilities in sensitive information systems such as banking systems and healthcare information systems [28]. In the case of IoT, where interconnected devices are used extensively in critical areas like healthcare and automated industry applications are more vulnerable than ever due to the interconnectedness of the systems devices and the various types of capabilities that each IoT application performs. Considering the diverse and varying constraints on these systems, applying static code analysis can ensure IoT systems accomplish what they are meant to do and perform their required functions, including maintaining the expected levels of performance and security. Static code analysis goes a long way to ensuring that these systems are reliable and effective; modern systems can be made effective and safe [29].

The interest in the use of LLMs for the enhancement of static code analysis has been inspired, in no small part, by existing and modern LLMs such as ChatGPT. The capabilities of LLMs in general are unique, as they are trained on large-scale, diverse datasets that contain source code and natural language [30]. LLMs can detect a range of issues in code, especially those that require contextual and complex reasoning. Furthermore, by synthesizing information across different parts of the code, LLMs can detect some of the root causes of the issues in the source code, which has broad implications for readability, maintainability, and software development best practices overall [31]. In fact, traditional static code analysis tools are weak to identify certain types of code issues compared to LLMs [32].

The rapidly evolving field of software engineering has the potential to experience tremendous changes in the field of software development with AI-based models such as LLMs. Generative AI models are widely adopted in the field, since there are clear benefits in terms of the efficiency of software development processes (with regards to productivity and development cycle speed). Industry experts predict that by 2027, around 70% of all professional software developers will use AI-assisted coding tools for routine coding tasks,

optimization, and debugging [33]. The future of the software industry holds a great potential for promoting the software industry with the use of LLMs for static code analysis, or even the combination of LLMs with existing traditional static code analysis tools. Given their knowledge of natural and programming languages, LLMs can find issues, provide code improvement suggestions, and thus improve a developer's productivity.

This chapter presents a systematic survey focused on LLM-based static code analysis. Through rigorous examination of many related studies, the survey provides insights into this rapidly evolving field as presented in the subsequent sections.

2.2 Systematic Literature Survey

2.2.1 Research Methodology

This survey follows established guidelines for conducting systematic literature surveys which were presented in [34]. Figure 2.1 illustrates the five steps of the employed methodology. The first step defines the purpose and scope of the thesis and defines its objectives along with the research questions to be answered. The second step, termed the search strategy, focuses on devising a method for searching relevant articles related to the topic under investigation. In the third step, the identified papers are screened and filtered. The fourth step involves data extraction, whereby the selected papers are reviewed and relevant data that meets the objectives of the research is collected. The results are documented during the fifth step of the process. Subsequently, the following sections explain these steps in detail.

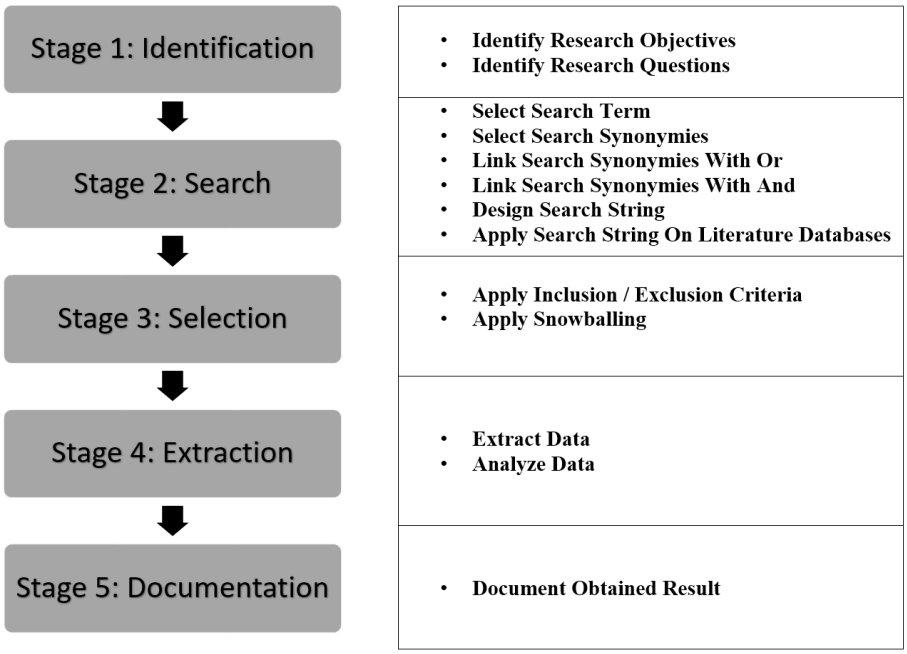

Figure 2.1: The employed methodology for literature survey

2.2.2 Identification of Research Objectives and Questions

1. Research Objectives: This systematic survey seeks to examine the published works on involving static code analysis with LLMs by searching, assessing, and categorizing state-of-the-art contributions. This is done so that developers and researchers can understand the answers to particular questions and subsequently enhance their efforts in development and research.
2. Research Questions (RQs): Several RQs have been defined and answered in this chapter. Each RQ deals with a different dimension of the topic of this thesis, as follows:
    - RQ1: What are the most frequently used LLMs for static code analysis tasks?
    - RQ2: What are the traditional static code analysis tools that are used to assess the quality of LLMs for static code analysis?
    - RQ3: What are the major programming languages used in research for specific static code analysis tasks that involve the use of LLMs?
    - RQ4: What is the range of software engineering activities that have been targeted by static code analysis with LLMs?
    - RQ5: Which evaluation metrics are most used in quantifying the accuracy and utility of LLMs in static code analysis?
    - RQ6: What prompt design strategies are most effective for optimizing LLMs performance in static code analysis?
    - RQ7: What are the common challenges and limitations in leveraging LLMs for static code analysis?

2.2.3 Search Strategy

1. Literature Sources: Well-known standard online databases such as IEEE Xplore, Elsevier Science Direct, and ACM Digital Library that index the papers relevant to the scope of this thesis were selected as literature sources.
2. Search String: Using the literature sources, the following search string was used to locate the relevant papers:

"(Generative AI OR LLM OR Large Language Model) AND (static code analysis)"

All terms of the search string were linked with each other using Boolean operators [35]. The Boolean "OR" was employed to link synonyms or related terms that refer precisely or broadly to different aspects of the thesis's topic and the Boolean "AND" was used to link the major terms.

2.2.4 Paper Selection

3. Paper Inclusion/Exclusion Criteria: A set of inclusion and exclusion criteria were established and employed to decide whether a paper is relevant to this thesis or not. These criteria, which are listed below, have been applied based on the titles, abstracts, and full text reading of the collected papers.
   a. Inclusion criteria:
      - Papers related directly to static code analysis using LLMs.
      - Papers published over the last two years (2023 - 2024). According to the search and exploration of the literature, papers on static code analysis using LLM started in 2023.
   b. Exclusion criteria:
      - Papers not published in English.
      - Papers not peer reviewed (e.g., grey literature).
      - Papers not published electronically.
      - Papers that are duplicates of other papers.
      - Papers without clear results and evidence.

   1. Snowballing: To reduce the risk of missing some relevant papers, the snowballing search technique [36] was applied to the remaining papers. In snowballing, the reference list of each paper is checked with the inclusion/exclusion criteria. Then, the paper selection process is applied recursively to the papers that have been found. Figure 2.2 shows the number of included and excluded papers at each stage of the paper selection process.

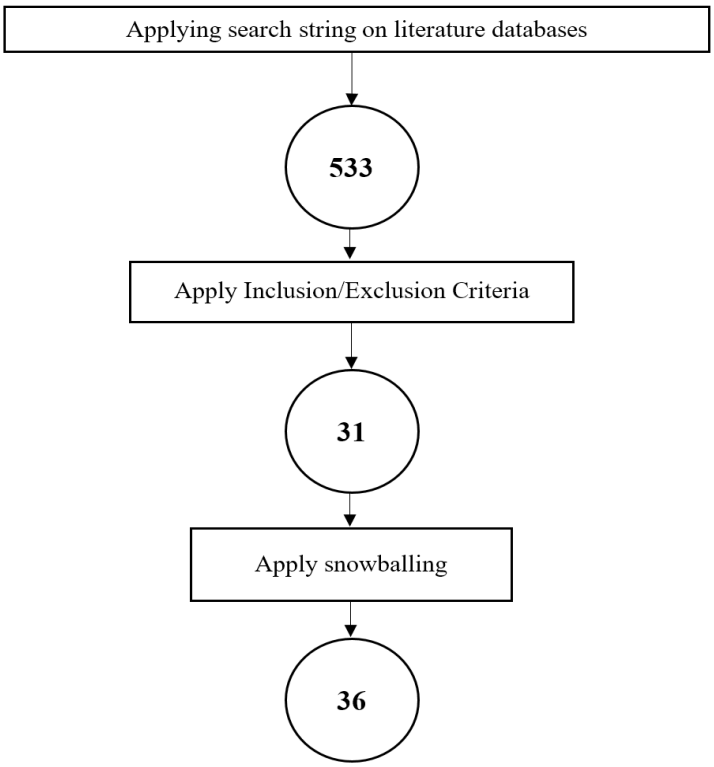

Figure 2.2: Results of paper selection process

All the final selected papers are listed below:

[33], [26], [6], [37], [38], [1], [3], [39], [40], [30], [41], [42], [43], [5], [44], [45], [46], [8], [47], [2], [48], [13], [49], [50], [31], [51], [25], [52], [53], [54], [12], [4], [55], [11], [23], [9].

2.2.5 Data Extraction and Analysis

To address the RQs, data were taken from the chosen papers and thoroughly examined. An Excel document with several fields made specially to include the extracted data. As presented in Table 2.1, each field contains a data item and a value.

Table 2.1: Data extraction form

Data Item	**Value**	**RQs**
Paper number	Paper ID.	None
Paper title	Title of the study.	None
Used LLMs	LLMs used in the studies.	RQ1
Static code analysis tools	Static code analysis tools used in the studies with LLMs for code evaluation.	RQ2
Programming languages	Programming languages involved in the studies utilizing LLMs for static code analysis.	RQ3
Type of tasks	Tasks of static code analysis performed in the studies using LLMs.	RQ4
Evaluation metrics	Metrics used to assess the effectiveness of LLMs for static code analysis.	RQ5
Prompt designs	Strategies of effective prompt designs for LLMs used in static code analysis tasks.	RQ6
Limitations and Challenges	Limitations and Challenges of performing static code analysis using LLMs.	RQ7

2.2.6 Literature Analysis

All of the papers selected were thoroughly examined in order to address the RQs that were identified for this thesis. Based on the results, each RQ is represented by a brief title and is covered in the next subsections.

2.2.6.1 Popular LLMs for static code analysis (RQ1)

Table 2.2 presents the most used LLMs in literature. OpenAI's ChatGPT-4 and ChatGPT-3.5-turbo models are leaders in the field, ChatGPT-4 being cited in 16 papers alone, showcasing the versatility and popularity of the model. ChatGPT models, particularly ChatGPT-4 and ChatGPT-3.5-turbo, are popular LLMs because they are the best performers at tasks, are easy to reach through OpenAI Application Programming Interface (API), and have better contextual understanding and reasoning. With researchers, working on AI powered static analysis and software engineering, they remain a top choice especially when economical options like ChatGPT-3.5-turbo are available [22], [52]. This is because the researchers can find fairly priced options for a variety of tasks including bug detection, code generation, and even vulnerability scans. LLMs are known to have a wide range of applications in static code analysis. They aid in enhancing error detection, warning verification, as well as static analysis tests translations across programming languages. LLMs are also known to enhance the Precision and efficacy by reducing false positives and false negatives in bug detection, malicious code detection, and even vulnerability detection. Their scope of usage expands within programming education for error checking and explanation purposes, and even cybersecurity for aiding in detection and remediation of vulnerabilities.

Table 2.2: LLMs used for static code analysis

SN	Used LLM	Corresponding Papers	Total Papers
1.	ChatGPT-4	[26], [6], [37], [38], [1], [3], [33], [39], [40], [30], [41], [42], [43], [5], [44], [45]	16
2.	ChatGPT-3.5-turbo	[46], [40], [30], [8], [5], [44], [47], [45], [2]	9
3.	ChatGPT-3.5	[26], [48], [38], [39], [13], [42], [43]	7
4.	CodeLlama	[26], [30], [49], [42], [50], [45]	6
5.	ChatGPT-2	[31], [13], [51]	3
6.	GitHub Copilot	[37], [3], [8]	3
7.	CodeGen	[3], [25], [47]	3
8.	ChatGPT-3	[31], [1]	2
9.	Mixtral	[6], [49]	2
10.	Google Bard	[37], [52]	2
11.	Codex	[53], [25]	2
12.	Polycoder	[53], [25]	2

13.	DeepSeek-Coder	[49], [50]	2
14.	WizardCoder	[50], [45]	2
15.	Mistral	[50], [45]	2
16.	ChatGPT-1	[31]	1
17.	StarCoder	[26]	1
18.	CodeFuse	[13]	1
19.	AI21 Jurassic-1	[53]	1
20.	BERT	[54]	1
21.	CODEDOCTOR	[12]	1
22.	INCODER	[12]	1
23.	GraphcodeBERT	[12]	1
24.	StarChat-Beta	[30]	1
25.	Phi-2	[50]	1
26.	Davinci	[47]	1
27.	Vicuna	[45]	1

2.2.6.2 Baseline static code analysis tools (RQ2)

Table 2.3 presents the use of different static code analysis tools outlined in the literature, with particular focus on their application and frequency. Programming Mistake Detector (PMD) emerges as the most widely cited tool, appearing in three papers, suggesting its effectiveness in code analysis and bug detection. SonarQube and CodeQL, cited in two papers, shows how useful it is for monitoring code quality. The tools Simian and Static Analyzer had only single mention due to their limited use. Tools like PyCG and HeaderGen, show the growing desire for language and domain specific static analysis, Python and C++ in this instance.

PMD is one of the leading static code analysis tools for many reasons. It is multi-language and analyzes Java, JavaScript, Apex and PLSQL which makes it very versatile and widely usable. Another reason is that PMD is extremely configurable, allowing developers to create new or change the old rules to comply with specific coding standards and best practices [3]. This ensures that every development team can tweak the functionality to meet their requirements. Also, PMD is an open-source project. This means that people can improve it, leading to even further improvements in sophistication. The ability to detect code duplication, dead variables, and potential bugs assists to improve the quality and longevity of the code. PMD also works with other popular build tools and IDEs which makes it easy to use in various development environments. Together, all these factors combined with an active community and frequent releases increase PMD's recognition as one of the best static code analysis tools.

Table 2.3: Static code analysis tools used in literature

SN	Used Tool	Corresponding Papers	Total Papers
1.	PMD	[48], [33], [4]	3
2.	SonarQube	[4], [5]	2
3.	CodeQL	[5], [3]	2
4.	Simian	[48]	1
5.	Custom static analysis tool	[55]	1
6.	Static Analyzer	[11]	1
7.	FindBugs	[4]	1
8.	Coverity	[4]	1
9.	TECA	[4]	1
10.	Rust-Code-Analysis	[23]	1
11.	Infer	[44]	1
12.	PyCG	[45]	1
13.	HeaderGen	[45]	1
14.	TypeEvalPy	[45]	1

2.2.6.3 Target programming languages (RQ3)

Table 2.4 presents the focus of research regarding the target programming languages for static code analysis using LLMs. The Java language was noted more often than other languages, as it appeared in 22 of the examined papers, while Python appeared in 17 and the C language in 14. Other languages like C++ had a moderate representation, 10 papers.

In comparison, Swift, Kotlin, and Go only appeared in 2 papers each, while niche/specialized languages, such as Solidity, Ruby, Rust, Verilog, PHP, Objective-C, SQL, Perl, Scala, and R all only appeared in 1 paper each. These findings illustrate the prominence of mainstream languages like Java and Python, and their prevalence in research studies, especially for the use of LLMs in bug detection, vulnerability detection, and code understanding through static code analysis. Java and Python are the two most studied programming languages for static code analysis due to their widespread use and prevalence in the industry. Java is frequently used in enterprise applications and on Android devices, while Python reigns supreme in data science, AI, and web apps [37]. There are well-featured static code analyzers for Java (e.g., PMD, SonarQube) and Python (e.g., Pylint, Bandit), with vibrant communities and considerable resources invested in them, raising the bar for quality of both languages. Java's object-oriented structure and static typing enables precise interpretation and analysis of the target program. In comparison, Python’s dynamic features create unique challenges that nurture the ingenuity of researchers. Also, the active interest from industry and academia, underlines the importance of these programming paradigms for static code analysis research. The limited scope of representation of the other general languages or domain specific languages suggests a

reasonable avenue for subsequent research regarding the potential of LLMs in using them.

Table 2.4: Programming languages used in literature

SN	Programming Language	Corresponding Papers	Total Papers
1.	Java	[26], [6], [48], [37], [52], [55], [1], [33], [39], [13], [40], [12], [25], [49], [42], [8], [4], [23], [5], [44], [51], [2]	22
2.	Python	[26], [48], [37], [52], [55], [38], [1], [3], [13], [53], [46], [12], [30], [43], [5], [47], [45]	17
3.	C	[26], [52], [1], [3], [53], [46], [40], [54], [30], [43], [4], [50], [47], [51]	14
4.	C++	[1], [46], [40], [12], [49], [8], [4], [50], [47], [51]	10
5.	JavaScript	[48], [37], [1], [40], [30], [43]	6
6.	C#	[11], [40], [49], [41]	4
7.	Swift	[1], [40]	2
8.	Kotlin	[1], [40]	2
9.	Go	[40], [47]	2
10.	Solidity	[26]	1
11.	Ruby	[1]	1
12.	Rust	[1]	1
13.	Verilog	[53]	1
14.	PHP	[40]	1
15.	Objective-C	[40]	1
16.	SQL	[40]	1
17.	Perl	[40]	1
18.	Scala	[40]	1
19.	R	[40]	1

2.2.6.4 Common static code analysis tasks (RQ4)

Numerous tasks in the collected papers have been identified regarding the static code analysis and applied LLMs are presented in Table 2.5. The greatest number of papers that is 12, has been published in the area of security weaknesses and the attempts that are done to discover and resolve these issues in the code. Static behavior analysis and code quality estimation and control are fundamental parts of 10 papers that concern the understanding of program behavior and the quality control standards, respectively. Also, 6 papers dedicated to bug detection highlighting the ongoing approach of seeking errors in different phases of software development and correcting them.

There are relatively few studies looking at other more specific issues like understanding syntax, detecting variable misuses, and reasoning for errors. These suggest very specific research areas. It also shows research activities like compliance with the green coding

initiative, breaches of naming conventions, and the reasoning logic behind error detection, all of which are performed by individual researchers. These definitional boundaries are among the newer and broader of the static code analysis challenges in which sustainability coding is being investigated and maintainability are being researched. The scope of static code analysis and its applicability to these varied domains of software engineering are evident in these varied task categories.

Table 2.5: Types of static code analysis tasks performed using LLMs

SN	Type of Task	Corresponding Papers	Total Papers
1.	Security vulnerability detection	[6], [3], [53], [46], [54], [41], [42], [43], [50], [47], [51], [2]	12
2.	Static behavior analysis	[26], [55], [30], [49], [43], [4], [23], [5], [44], [45]	10
3.	Code quality assurance	[48], [55], [33], [39], [13], [12], [8], [4], [23], [5]	10
4.	Bug detection	[52], [33], [53], [25], [49], [9]	6
5.	Syntax understanding	[26]	1
6.	Variable misuse detection	[11]	1
7.	Adherence to green coding rules	[37]	1
8.	Logical reasoning in error detection	[38]	1
9.	Identifying violations (naming conventions)	[12]	1

2.2.6.5 Evaluation metrics (RQ5)

The common metrics used to evaluate the capabilities of LLMs for static code analysis are presented in Table 2.6. Accuracy is the most reported metric, found in 11 papers, showing how valuable this metric is for the effectiveness of these models in specific cases. Other metrics were used in the following order of importance: Precision, F1-Score, manual verification and False Positive Rate in problem and warning capture. Metrics like Recall, match rate and True Positive Rate show the sensitivity of the models to catch important problems [53]. Domain specific metrics are those sustainability metrics like Green Code Compliance Percentage, Rule Satisfaction Rates, Security Correctness. Syntax error counts, repair success rates and functional correctness prove the value of evaluating code quality and the effect of automatic repair of codes. Complexity metrics, like Cyclomatic Complexity, Cognitive Complexity, and Weighted Methods per Class assess the structural integrity and maintainability of the code. Different codes are compared and evaluated using unique metrics like DiffBLEU score, Levenshtein Distance and Jaccard Index. In terms of usage frequency, the most widely used means of measurement frequency is for the F1-Score and Accuracy which enable a more

holistic view of the metrics. Accuracy is used to define the number of correctly identified issues (True Positives + True Negatives) across all predictions which provides a comprehensive perspective on a tool's performance.

However, since the static code analysis dataset issue (positives) vs non-issue (negatives) is class-imbalanced, Accuracy cannot be relied upon as a metric since a high score could be achieved simply by predicting the majority class (non-issue) where no issues were found [25]. The F1-Score accounts for this since it is a combination of Precision and Recall [50]. The Precision metric defines the ratio of actual problem instances correctly identified from total expected problem instances to address false positives; Recall defines the ratio of correctly detected real issues to prevent false negatives. The F1-Score is the harmonic mean of Precision and Recall; this is a particularly interesting measure because it balances the two, which is important in static analysis tools since both types of errors (false positives and negatives) can produce major consequences. The integrated use of accuracy and the F1-Score together can be used to assess both the reliability and effectiveness of static code analysis. The range of metrics applied shows both the extensive research in static code analysis and its broad application to every field of software engineering.

Table 2.6: Evaluation metrics used in literature

SN	Metric	Corresponding Papers	Total Papers
1.	Accuracy	[26], [52], [38], [1], [11], [46], [54], [50], [44], [51], [2]	11
2.	F1-Score	[26], [38], [46], [49], [50], [44], [47], [51]	8
3.	Manual Verification	[6], [52], [33], [13], [41], [8], [5]	7
4.	Precision	[46], [49], [43], [50], [44], [47], [51]	7
5.	False Positive	[11], [42], [4], [50], [5], [47]	6
6.	Recall	[46], [50], [44], [47], [51]	5
7.	Match rate	[40], [30], [25], [49], [45]	5
8.	Number of rule violations	[48], [39], [5]	3
9.	Time to fix issues	[33], [41], [42]	3
10.	True Positive	[42], [4], [47]	3
11.	Syntax error counts	[55], [44]	2
12.	Success rate of LLM	[33], [43]	2
13.	False Negative	[11], [47]	2
14.	Security correctness	[53], [42]	2
15.	Soundness	[45]	1
16.	Completeness	[45]	1
17.	Jaccard Index	[26]	1

18.	Number of Prompts	[48]	1
19.	Green Code Compliance Percentage	[37]	1
20.	Rule satisfaction rates	[55]	1
21.	Plagiarism detection	[55]	1
22.	Pass@k	[38]	1
23.	Number of discovered vulnerabilities	[3]	1
24.	Top-k Accuracy	[50]	1
25.	Cyclomatic complexity	[39]	1
26.	Cognitive complexity	[39]	1
27.	Correctness Metrics	[13]	1
28.	Coverage Metrics	[13]	1
29.	Functional correctness	[53]	1
30.	Repair success rate	[53]	1
31.	BLEU	[40]	1
32.	Mean	[12]	1
33.	Bug/Patch ratio	[25]	1
34.	Logical Lines of Code	[8]	1
35.	Number of Statements	[8]	1
36.	McCabe Cyclomatic Complexity	[8]	1
37.	Nesting Level	[8]	1
38.	Coding smells	[8]	1
39.	ABC metric	[23]	1
40.	Weighted Methods per Class	[23]	1
41.	Number of Public Methods	[23]	1
42.	Number of Public Attributes	[23]	1
43.	Class Operation Accessibility	[23]	1
44.	Class Data Accessibility	[23]	1
45.	Area Under the Curve	[51]	1

2.2.6.6 Common prompting strategies (RQ6)

Table 2.7 categorizes various prompting strategies for LLMs across the collected papers. Structured Prompting is the most common with 8 papers, because it instructs LLMs without details of the problem at hand. Standardized or Basic Prompting is employed in 7 papers, which suggests how commonplace this simple form of prompting is. Few-shot Prompting is referenced in 6 papers and Zero-shot Prompting in 4 papers. Slightly more unusually, Iterative Prompting also has 4 papers and One-Shot Prompting has 3, suggesting some degree of interest in tailoring prompting to tasks, or prompting for step-wise, iterative reasoning. There are 2 papers each on Task-Specific Prompting, Chain-of-Thought (CoT) Prompting, Natural Language Descriptions, and High-Level Query Language. Other types of prompting, such as Temporal and Spatial Context, Scenario-Based Prompting, and Correction Prompts all have 1 paper each, indicating increasingly specialized (or perhaps emerging) research directions with respect to LLM prompting techniques in the case of static code analysis.

Structured Prompting, Standardized/Basic Prompting, and Few-shot Prompting are the most frequently used as a result of their effectiveness, ease of use, and their suitability to the static code analysis unique challenges. Structured Prompting is useful because it offers a coherent, systematic framework for directing models in code analysis, which corresponds effectively with the ordered characteristics of programming languages. This breakdown of more complicated tasks is useful for the model to become more successful in spotting syntax errors, code smells, or security vulnerabilities [12]. Standardized/Basic Prompting is chosen for its ease of use, as it involves straightforward instructions that can be applied from codebase to codebase, and analysis tasks of all sorts [41]. Few-shot Prompting is a trustworthy standard and exceptionally effective for evaluating static code analysis because it allows models to learn from limited examples which makes it very useful when datasets are small or when shifting between coding languages or styles. These three prompting strategies therefore offer the necessary Precision, scalability, and adaptability that is necessary for impactful static code analysis, making them most common approaches in the research context. This distribution shows the scope and distinctiveness of prompt engineering strategies that are being studied in the related works.

Table 2.7: Distribution prompting strategies

SN	Prompt Type	Corresponding Papers	Total Papers
1.	Structured Prompting	[48], [53], [30], [25], [41], [5], [47], [2]	8
2.	Standardized/Basic Prompting	[38], [46], [40], [30], [49], [42], [8]	7
3.	Few-shot Prompting	[26], [1], [3], [33], [42], [44]	6
4.	Zero-shot Prompting	[26], [1], [33], [44]	4
5.	Iterative Prompting	[48], [13], [9], [45]	4
6.	One-shot prompting	[33], [44], [45]	3
7.	Task-Specific Prompting	[37], [50]	2
8.	CoT Prompting	[38], [46]	2
9.	Natural Language Descriptions	[13], [25]	2
10.	High-Level Query Language	[55], [49]	2
11.	Temporal and Spatial Context	[49]	1
12.	Scenario-Based Prompting	[40]	1
13.	Correction Prompt	[38]	1

2.2.6.7 Common limitations and challenges (RQ7)

Several challenges and constraints on the use of LLMs for static code analysis were identified, as follows:

1. False Positive Rates: In the static code analysis, LLMs tend to produce many false positives that require a thorough human verification to detect actual issues. Some of the related challenges include how to detect Null Dereferences and Resource Leaks among others [3], [4], [5], [44], [47].
2. Token and Context Limitations: LLMs have limitations on the input size which makes them not very effective in handling large codebases or complex dependencies at a time. For example, dealing with imported module code or checking through large files is challenging [26], [1], [33], [39], [53], [12], [30], [41], [42], [5], [44].
3. Data Limitations: It is a common issue to find that the quality and diversity of training datasets directly impact the model's generality and accuracy. The reliance on databases such as Common Vulnerabilities and Exposures (CVE) which are prone to errors, decreases the accuracy of vulnerability detection [26], [37], [38], [11], [46].
4. Non-deterministic Outputs: Uncertainty about how many times LLMs will produce different outcomes from multiple executions for the same input creates problems when integrating them into static code analysis frameworks. For instance, making coding integration and development workflows deterministic is difficult due to the variability of results [37], [49].
5. Computational Costs: Resource-intensive nature is itself an issue regarding large-

scale code analysis; and the fine-tuning requirement makes it worse [49], [47], [45].

6. Model Hallucinations and Assumptions: Sometimes LLMs give wrong and overconfident results because of built-in biases about the underlying data and context. In bug detection, for instance, they usually hallucinate and forget dependencies [26], [42], [44].
7. Scalability and Adaptability Issues: LLMs do not generalize across languages, ecosystems, and they also do not learn complex design patterns. For example, they cannot scale in retaining language semantics during cross-language analyses [52], [54].

2.3 Comparison with Related Studies

This section presents the most related works to this thesis. LLMs were evaluated on whether they can detect software vulnerabilities in [9], [56]. This includes evaluating LLMs against traditional tools like Snyk and Fortify, or other pretrained models for detecting vulnerabilities. LLMs were used to perform static taint analysis, using the data entering the program to judge whether paths are vulnerable True or False Positives by contextualizing, as in [57]. The use of LLMs goes beyond the detection of software vulnerabilities to other tasks of software vulnerability analysis, such as vulnerability assessment, vulnerability location, and vulnerability description, as the extra context can aid the LLM in analyzing vulnerabilities [50]. LLMs were used to identify and address general code quality problems [48]. This covers both error detection and correction (such as syntax and type errors). By producing and ranking code fixes to increase acceptance rates and reduce developer workload, LLMs assisted in addressing code quality issues with the use of tools such as CodeQL and SonarQube in the context of static code analysis [1].

Additionally, LLMs were used to refactor and fix bugs found by static code analysis tools, especially those related to design, error proneness, best practices, and code style [3]. Although LLMs are good at detecting issues in single locations, they might not be able to grasp the larger code context for intricate design problems [6]. To determine whether potential bugs found by static code analysis are real bugs or False Positives, LLMs might be consulted, particularly in circumstances that are unclear [32]. To lower the rate of False Positives, tools such as SkipAnalyzer combine automatic bug repair, false-positive alerts, and LLM-based static bug detection [5]. Assessing the efficacy of LLMs to accomplish program analysis tasks like data dependency analysis, taint analysis, and pointer analysis is critical for their use alongside static code analysis [10]. For LLMs to be usefully and reasonably deployed for code-related tasks, it is essential to recognize how well LLMs can understand program syntax and

the static approximation of program behavior. Even if LLMs are promising for the domain of type inference, call graph analysis tasks may be best addressed with traditional tools [47]. To summarize, the tasks being assigned to LLMs range from low-level code comprehension and static behavior approximation, to high-level tasks like detecting and correcting numerous types of violations, and, most importantly, refining the output of established, traditional static code analysis. Although there have been previous research works that assess the capabilities of LLMs in the areas of software vulnerability detection, taint analysis, and in vulnerability detection and assessment in general, this thesis extends the literature by assessing LLMs in the specific area of coding violations in Python; a significant area of static code analysis but one that has received little attention in the literature. Table 2.8 summarizes the key differences between existing studies and this thesis.

Table 2.8: Summary of comparison with related studies

Study	Study Focus	Used LLMs	Target Language	Prompt Types	Baseline Tools	Used Dataset	Evaluation Metrics	Results
[1]	Code Quality violations (maintainability, readability, security)	GPT-3.5-Turbo, GPT-4	Python, Java	Template-based and Scoring Rubric	Sorald	CodeQueries, Sorald benchmark	%Files fixed, issues remaining, human acceptance rate.	CORE successfully generated static-check-passing code revisions for 88.8% of Python files and 82.2% of Java files, functioning comparably to rule-based APR tools but with significantly less engineering effort.
[3]	General Code Quality Improvement	ChatGPT (GPT-3.5/4)	Java	Zero-shot, One-shot and Few-shot	PMD	10 open-source Java projects	Issue acceptance/rejection ratios, qualitative student feedback.	ChatGPT successfully helped students resolve many PMD warnings, though 'Design' violations took longer to fix due to LLM context limitations.
[5]	Bug detection and vulnerability detection	ChatGPT-3.5 Turbo, ChatGPT-4	Java	Zero-shot, One-shot, Few-shot, and CoT	Infer, VulRepair, Logistic Regression	10 open-source projects	F1-Score, Accuracy, Precision, Recall, Logic/Syntax Rate.	SkipAnalyzer improved detection precision by 12.86% (Null Dereference) and 43.13% (Resource Leaks) over Infer, and generated correct logical patches for over 91% of bugs.
[6]	Bug Detection	GPT-3.5, GPT-4	C	CoT, Task Decomposition and Progressive Prompts	UBITect	20 randomly selected false positive false negative cases from UBITect's	Soundness, Completeness, False Negative Detection.	Pruned 16 out of 20 false positives in a pilot study by using ChatGPT to generate precise function summaries that bypass limitations in UBITect's symbolic execution.
[9]	Vulnerability Detection	CodeGen-2B, Davinci, GPT-3.5-Turbo	C, C++, Python, Go	Zero-shot	Flawfinder, RATS, Checkmarx,	Code Gadgets (61,638 snippets); CVEfixes	False Positive Rate, False Negative Rate, True Positive	LLMs achieved high recall for detecting SQLi and Buffer Overflows but suffered from very high false positive rates compared to traditional static

					VulDeePecker		Rate, Precision, Recall, F1-Score.	tools and custom neural networks.
[10]	Static Behavior Understanding	GPT-4, GPT-3.5, StarCoder, CodeLlama -13B	C, Java, Python, Solidity	Role-based and Instruction-based includes Zero-shot and Few-shot	Tree-Sitter, Slither, Frama-C, MutantBench	2,560 code samples (created using analysis tools)	F1-Score, Jaccard Index, Accuracy.	LLMs (especially GPT-4) understand code syntax (ASTs) well but struggle with deeper static behaviors like pointer and taint analysis, showcasing data-shift vulnerabilities.
[47]	Python Static Analysis	26 LLMs (GPT, LLaMA, Mistral, etc.)	Python	One-shot	PyCG, HeaderGen, HiTyper	Micro-benchmarks from PyCG, HeaderGen, TypeEvalPy	Completeness, Soundness, Exact Matches.	Traditional static tools (PyCG, HeaderGen) heavily outperformed LLMs in callgraph analysis, but LLMs (like GPT-4) significantly outperformed traditional tools in type inference tasks.
[48]	Error and bug Detection	GPT-3.5, GPT-4	Python	CoT	None	169 buggy snippets based on HumanEval	F-score, Accuracy	ChatGPT demonstrated low correlation between error detection and formal static analysis difficulty; utilizing Chain-of-Thought (CoT) prompting did not significantly improve code correction abilities.
[50]	Vulnerability Detection	DeepSeek-Coder, CodeLlama , StarCoder, WizardCoder, Mistral, Phi-2	C/C++	Few-shot and Task Description	Devign, Reveal, IVDetect, LineVul, SVulD	Big-Vul (Real-world C/C++ vulnerabilities)	F1-Score, Recall, Precision, Accuracy, False Positive Rate, Top-k Accuracy	Fine-tuned LLMs generally performed weaker than transformer-based static approaches for vulnerability detection, but exhibited strong performance on vulnerability assessment when supplied with detailed contextual information.
[56]	Vulnerability Detection	GPT-4, DaVinci, Curie, Ada	C, Ruby, PHP, Java, JS, C#, Go, Python	System role	None	129 code samples from NASA/DoD repositories	Vulnerability count, False Positive Rate,	GPT-4 identified 4x more vulnerabilities than Snyk and successfully provided patches, reducing high-severity vulnerabilities by 94% with minimal code bloat.
[57]	Static Taint Analysis	GPT-4, GPT-3.5, Llama 3, DeepSeekCoder	Java	Few-shot, Zero-shot, and Contextual prompts	CodeQL, Facebook Infer, SpotBugs, Snyk	CWE-Bench-Java: 120 manually validated vulnerabilities	#Detected vulnerabilities, Average False Discovery Rate, Average F1-Score, Recall, Precision	IRIS + GPT-4 detected 55 vulnerabilities (28 more than CodeQL) and successfully reduced the false discovery rate by 5.21% by contextually inferring taint specifications.
[32]	Detecting Use-Before-Initialization (UBI) bugs in the Linux Kernel	GPT-4, GPT-3.5, Claude 2, Bard	C	Few-shot, Progressive Prompt, Task Decomposition, Self-Validation, CoT	UBITect	Random 1,000 inconclusive potential bug reports from Linux Kernel v4.14	Precision, Recall, Accuracy, F1-Score, Consistency	Identified 13 previously unknown UBI bugs in the Linux kernel with 50% precision and 100% recall on known bugs by using LLMs to drive post-constraint guided path analysis.

This Thesis	Python coding violation detection	ChatGPT, Gemini, Claude, DeepSeek, Kimi, Qwen	Python	Structural Prompt, CoT Prompt, Role-based Prompt	Pylint, Flake8	75 Python code segments from LeetCode, spanning Easy, Medium, and Hard levels, were manually labeled with 27 common PEP-8 violations	Precision, Recall, F1-Score, FDR, CR, PA	Claude Sonnet achieved the best performance, outperforming Flake8, while ChatGPT and Pylint demonstrated the weakest results. The Thesis also found that structural prompting and individual code segment processing improved overall performance and result stability.

While previous study, for example, has focused in evaluating LLMs in comparison to security-focused tools like Snyk and Fortify, this thesis evaluates the LLMs in comparison to traditional Python static code analysis tools (i.e., Pylint and Flake8), with an emphasis on coding quality. Furthermore, where previous study has focused much attention on the ability of LLMs to correct errors while reducing the number of False Positives among existing findings, this thesis focuses on presenting a new taxonomy of 27 Python-specific coding violations and systematically evaluates six state-of-the-art LLMs (i.e., ChatGPT, Gemini, Claude, DeepSeek, Kimi, and Qwen) under different prompting strategies (i.e., Structural Prompt, CoT Prompt, and Role-based Prompt), not only LLMs' strengths in detecting certain violations, but also their weaknesses due to failures in semantic reasoning and consistency; a contribution that better assesses the proposed use of LLMs in coding quality assurance. Finally, the knowledge gained regarding batch and individual code evaluation as well as prompting techniques offers practical insights for integrating LLMs in developer workflows rather than remaining a theoretical suggestion, as was proposed often in previous studies.

Chapter Three
Research Methodology

Chapter 3: Research Methodology

3.1 Introduction

This chapter presents the methodology used to evaluate LLMs in detecting Python coding violations. The employed methodology is guided by five research questions addressing performance comparison with static code analysis tools, violation-type detection, model consistency, prompting strategies, and batch versus individual code analysis. A dataset of 75 Python code segments, manually annotated according to PEP-8 and best coding standards, was used as the ground truth. PEP-8 was selected because it is the official and widely accepted style guide for Python, providing standardized conventions for readability, maintainability, and code quality. Six state-of-the-art LLMs were compared against Pylint and Flake8 using three prompting strategies: structured, chain-of-thought, and role-based. Performance was assessed using standard metrics such as Precision, Recall, F1-Score, and consistency-related measures, providing a solid foundation for the results and analysis presented in the next chapter.

3.2 Research Methodology

The employed research methodology consists of several steps for conducting the experiments (shown in Figure 3.1); identifying research questions, preparing Python code dataset, manually labeling data, selecting LLMs and static code analysis tools, designing prompts for the used LLMs, defining evaluation metrics, and analyzing results to assess LLMs' ability to detect Python coding violations. Each step is detailed in the following sections:

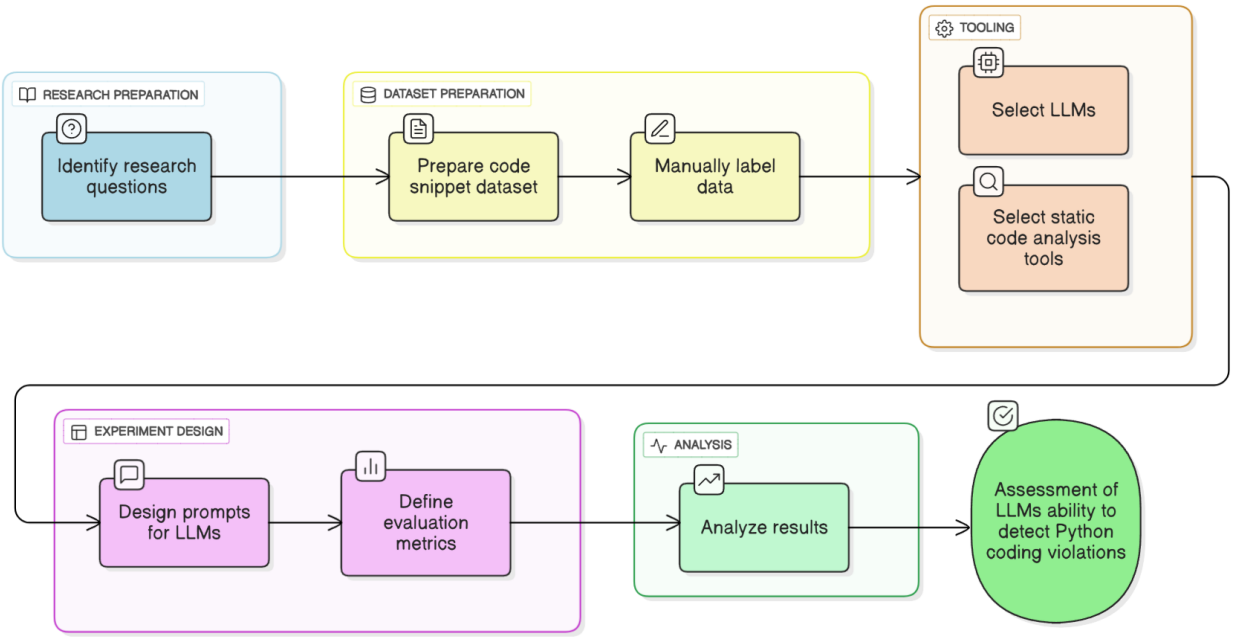

Figure 3.1: The experimental research methodology of this thesis

3.3 Research Questions

For the experimental part of this thesis, several RQs that address different aspects of the thesis's topic have been identified to be answered, as follows:

- **RQ1 (Performance Comparison):** How effective are LLMs at identifying Python coding violations compared to traditional Python static code analysis tools?
- **RQ2 (Violation-Type Detection Strengths and Weaknesses):** What types of Python coding violations are LLMs better at detecting compared to Python static code analysis tools, and which types do they struggle with?
- **RQ3 (Model Consistency):** How consistent are different LLMs in identifying the same Python coding violations?
- **RQ4 (Prompting Strategies):** Do different prompting strategies affect an LLM's ability to identify Python coding violations?
- **RQ5 (Batch vs. Individual Code Analysis):** Does analyzing Python code segments in batches, as opposed to individually, lead to different outcomes in the identification of coding violations?

3.4 Used Dataset

In this thesis, 75 Python code segments were extracted from LeetCode [3], the world's leading online programming learning platform that covers a broad range of programming topics, across three difficulty levels: Easy (25 segments), Medium (25 segments), and Hard (25 segments). The selected code segments ranged from 2 to 36 lines in length. Each code segment represents a programming task and the tasks of the dataset include data structures (arrays, strings, linked lists, trees, and graphs), algorithmic paradigms (dynamic programming, backtracking, greedy algorithms, and divide-and-conquer), and problem-solving techniques (two-pointers, sliding windows, and bit manipulation), which are used in almost all types of software and hardware systems. Then, all code segments were manually labeled by the authors for quality with different violations based on the PEP-8 and best coding standards and included 27 common violation categories as presented in Table 4.3. Examples include stylistic violations (e.g., missing whitespace), documentation issues (e.g., absent function Doc-strings), and structural/design flaws (e.g., nested methods). The full dataset and other materials are publicly available in a GitHub repository [4].

[3] https://leetcode.com/
[4] https://github.com/CodingWithLLMs/Online-CodeBase

Below is a sample Python code with different violations from the used dataset that demonstrates several violations of coding, their details are presented in Table 3.1.

```
def combinationSum(candidates,target):
    result=[]
    def backtrack(start,path,remaining):
        if remaining==0:
            result.append(path)
            return
        for i in range(start,len(candidates)):
            if candidates[i]>remaining:
                continue
            backtrack(i,path+[candidates[i]],remaining-candidates[i])
    backtrack(0,[],target)
    return result
```

Table 3.1: Different identified coding violations

#	Violation	Frequency
1.	Function name not snake case	2
2.	Missing whitespace after comma	8
3.	Function type hints Not written	2
4.	Function doc string Not written	2
5.	Missing Space around operators	3
6.	Expected blank line before a nested definition	1
7.	Add meaningful comments to logic	1
8.	Method inside method (no single responsibility)	1
9.	Functions input not Validate early	1
	Total identified violations	**21**

3.5 Used LLMs

To assess the capabilities and limitations of LLMs in detecting different Python coding violations, six state-of-the-art models were selected: DeepSeek-V3[5] (DeepSeek) from DeepSeek, ChatGPT 4-Turbo[6] (ChatGPT) from OpenAI, Claude 3.7-Sonnet[7] (Claude) from Anthropic, Gemini 2.5-pro-preview-03-25c[8] (Gemini) from Google, Kimi[9] from Moonshot AI and Qwen 2.5-Max[10] (Qwen) from Alibaba. These various LLMs, from different organizations,

[5] https://www.deepseek.com/
[6] https://chatgpt.com/
[7] https://claude.ai/
[8] https://gemini.google.com/
[9] https://www.kimi.com/
[10] https://chat.qwen.ai/

were chosen to ensure a comprehensive evaluation of their capabilities for static code analysis. Few of them have demonstrated proven effectiveness in code understanding, logical reasoning, and natural language processing tasks. Moreover, all selected LLMs are accessible freely through public user interfaces that developers typically use for direct code analysis, which makes them practical for conducting experiments.

3.6 Used Baseline Static Code Analysis Tools

To ensure rigorous evaluation of code quality, two well-known Python static code analysis tools were employed as baseline tools: Pylint [58] and Flake8 [59]. These tools complement each other. Flake8 checks for adherence to strict PEP-8 compliance and Pylint checks for other general software engineering and coding best practices. Thus, using both of these tools offers a decent selection for code analysis. In addition, both tools are well-known, have been around for a while, and produce results that the Python community trusts based on download statistics aggregated by PyPI Stats [11] [12]; thus, they offer a valid baseline for such analysis.

3.7 Used Prompt Types

The prompt type is a significant factor for LLM performance [46]. In this thesis, three types of common prompting approaches were applied to test the performance of the LLMs, as follows:

1. Structural Prompt (P1): Selected as a base prompt, which is a direct and task-focused prompt type that presents the coding problem without details. It tests an LLM's ability to understand requirements from minimal instructions, reflecting real-world scenarios with limited guidance [53]. Here is an example of structural prompt:

   "Analyze the following Python code for any PEP-8 violations or potential issues".

2. Chain of Thought (CoT) Prompt (P2): This type promotes stepwise reasoning, by explicitly requesting intermediate steps, breaking problems into sub-tasks, and including more thinking for evaluating an LLM's ability for clear and logical problem solving [48], [49], [60]. Here is an example of CoT prompt:

   "Analyze the provided Python code for PEP-8 violations, potential issues, or areas of improvement. Use Chain of Thought (CoT) approach to systematically check the code".

[11] https://pypistats.org/packages/flake8
[12] https://pypistats.org/packages/pylint

3. Role-based Prompt (P3): This type assigns expert roles and emphasizes production-quality outputs to test an LLM's adaptability to professional scenarios [39], [49], [53]. Here is an example of role-based prompt:

   "As a software engineer, you are tasked with analyzing the following Python code for any PEP-8 violations or potential issues".

The three aforementioned types of prompts were selected due to their proven ability to enhance LLM performance. Structural prompts improve clarity and relevance with task descriptions, output format, and required code context [53]. CoT prompts empower LLMs to address complex tasks via reasoning and problem solving [6]. Lastly, Role-based prompts offer LLMs an identity to follow (e.g., "as a software engineer") and a context for task-specific LLMs that focus the model on its task and improves the performance for the generation of outputs [38]. Beyond their functional differences, these prompting strategies were selected to represent varying levels of popularity in the literature. P1 represents the most commonly used baseline strategy, P2 reflects a less frequently adopted but emerging approach, and P3 was included as a novel strategy absent from the surveyed literature. This selection enabled the study to evaluate both established and unexplored prompting techniques and compare their effectiveness in producing professional-quality outputs.

3.8 Used Evaluation Metrics

To address the RQs of this thesis, a set of well-known evaluation metrics were employed including Precision (to measure how well a model avoids false positives), Recall (to measure how well a model finds all actual positives), and F1-Score (to measure the harmonic mean of Precision and Recall) [25], [45]. Other metrics such as False Discovery Rate (FDR) (to measure the proportion of false positives among all predicted positives.) [57], Consistency Rate (CR) (to measure total agreement rate for LLMs response), average CR [61], and Prompt Agreement (PA) (to measures LLM consistency by evaluating responses to varied versions of the prompt) [62] were also utilized.

- To answer RQ1 (Performance Comparison): After obtaining the results from the LLMs and static code analysis tools (processing 25 code segments at a time), each prediction was manually labeled as True Positive (TP), False Positive (FP), or False Negative (FN) based on the ground truth. Then Precision, Recall, F1-Score, and FDR were calculated for the entire dataset, as defined in Equations (3.1), (3.2), (3.3) and (3.4).

$$\text{Precision} = \frac{\text{TP}}{\text{TP+FP}} \quad (3.1)$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP+FN}} \quad (3.2)$$

$$\text{F1} - \text{Score} = 2 * \frac{\text{Precision} * \text{Recall}}{\text{Precision+Recall}} \quad (3.3)$$

$$\text{FDR} = \frac{\text{FP}}{\text{FP+TP}} \quad (3.4)$$

Where:

- TP: Correctly predicted positive case.
- FP: Incorrectly predicted positive when the actual is negative.
- FN: Incorrectly predicted negative when the actual is positive.

- To answer RQ2 (Violation-Type Detection Strengths and Weaknesses): The results were filtered by classification (FN and TP) and grouped by type and subtype to assess the LLM's strengths and weaknesses in detecting violation types.
- To answer RQ3 (Model Consistency): The obtained results were filtered to find cases with both (FN and TP) for each LLM, enabling consistency assessment. Average CR was computed across three prompts (P1, P2 and P3) to evaluate performance stability, and PA was calculated to identify the LLM with the highest agreement rate, as defined in Equations (3.5) and (3.6).

$$\text{Average CR} = \frac{1}{n}\sum_{i=1}^{n} \text{Recall } i \quad (3.5)$$

$$\text{PA} = \frac{\text{\# Prompts with} \geq 1 \text{ TP}}{\text{Total Prompts}} \quad (3.6)$$

Where:

- n: Total number of prompts.

- To answer RQ4 (Prompting Strategies): Three prompting strategies structured, CoT, and role-based were evaluated using Precision, Recall, and F1-Score to assess their impact on the LLMs' performance.
- To answer RQ5 (Batch vs. Individual code Analysis): 15 code segments (five per difficulty level) were randomly selected and evaluated using only the base prompt (P1) to check if individual segment assessments differ across all LLMs. Then, model predictions were manually annotated as TP, FP or FN by comparing them to the ground truth. Using this, Precision, Recall, and F1-Score were calculated for the (P1) dataset and compared these results with those from (RQ1) to identify any discrepancies or consistencies. It is worth mentioning that RQ5 is a comparative research question focusing on processing mode effects rather than full-dataset performance evaluation, where a subset of code segments was used to mitigate the impact of increased context

length in batch processing, including reduced focus on individual code segments (i.e., attention dilution) and interference between multiple code segments within the same prompt (i.e., context interference).

Chapter Four
Results and Discussion

Chapter 4: Results and Discussion

4.1 Introduction

In this chapter, the experimental results of this thesis are presented and discussed. Each RQ is summarized with a short title and discussed in its respective section based on the obtained results.

4.2 Performance Comparison (RQ1)

The obtained results for this RQ, based on three sessions per LLM using (25 code segments each with base prompt (P1)), are shown in Figure 4.1. Claude achieved the highest F1-Score (0.81), demonstrating a strong balance between Precision (0.99) and Recall (0.69), closely followed by Flake8 with F1-Score (0.79), which maintained perfect Precision (1.00) but slightly lower Recall (0.66). Gemini also performed well with F1-Score (0.78) with near-perfect Precision (0.99) and moderate Recall (0.64). On the contrary, Kimi with F1-Score (0.56), DeepSeek with F1-Score (0.46), and Qwen with F1-Score (0.36) showed declining performance due to having lower Recall (0.39), (0.34), (0.23), respectively, while having relatively high Precision. Notably, the worst-performing by far were Pylint and ChatGPT, with F1-Scores (0.35) and (0.13), respectively, primarily due to their extremely low Recall (0.21 and 0.07) despite being able to maintain perfect or near-perfect Precision. These results reveal that some LLMs, like Claude, can do better than traditional static analysis tools. They also suggest that some LLMs, such as Claude, can achieve reasonably good results with a direct prompt type with minimal instructions/details.

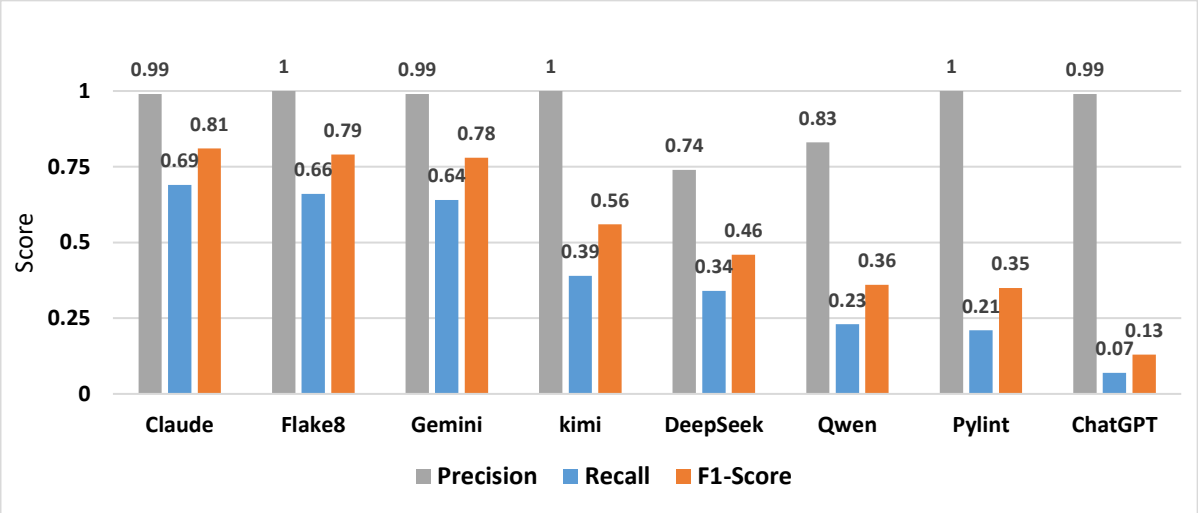

Figure 4.1: Performance comparison of LLMs vs. static code analysis tools

Another critical aspect for the overall performance of the LLMs is FDR, as seen in the different reliability results for the LLMs for all three prompts (P1, P2, and P3) in Table 4.1. Per-prompt FDR results are reported to show prompt-specific performance, followed by the

average FDR as an overall summary measure of model performance across different prompting strategies. It can be seen that Kimi was the top-performing model with a perfect FDR of (0.00) across all prompt types, reflecting its stabile Precision. Claude and Gemini were also remarkably reliable, with both models achieving an average FDR of only (0.01), with the added advantage of Claude achieving a perfect score of (0.00) for the third prompt type. ChatGPT was also reliable, averaging (0.06), though FDR increased to (0.17) with the third prompt type, reflecting some prompt sensitivity. DeepSeek had higher average FDR rates across the board, with scores ranging between (0.26) and (0.35), averaging a score of (0.29), and reflecting its reliability regarding FP. The most variable model was Qwen, as it achieved FDR scores ranging between (0.00) for the second prompt type to (0.92) for the third prompt type, averaging a score of (0.36), indicating its highly variable performance. These findings suggest that while models such as Kimi, Claude and Gemini are suitable for tasks that require reliability regardless of prompt use, others such as Qwen may require more careful attention to avoid False Discoveries. This only reinforces how careful the consideration of model and prompts was needed in tasks that had such a high variability in their performance, as reliability was needed across all prompt types.

Table 4.1: LLMs' FDR across three prompt type

#	LLMs	FDR			Average FDR
		Round-1-(P1)	Round-2-(P2)	Round-3-(P3)	
1.	Qwen	0.17	0.00	0.92	0.36
2.	DeepSeek	0.26	0.35	0.27	0.29
3.	ChatGPT	0.01	0.01	0.17	0.06
4.	Gemini	0.01	0.00	0.02	0.01
5.	Claude	0.01	0.02	0.00	0.01
6.	Kimi	0.00	0.00	0.00	0.00

4.3 Violation-Type Detection Strengths and Weaknesses (RQ2)

Here, the obtained result from three prompt types (P1, P2, and P3) were used to determine the effectiveness of the LLMs and static code analysis tools at identifying (27) coding violations within the code segments that were used, as well as distinguishing correct versus incorrect identifications, as shown in Figure 4.2. The results indicate that the effectiveness of the tested models varied, with the most successful models being both Claude and Gemini, which both achieved (16) correct identifications and (11) incorrect. The next most successful were Flake8, which achieved (15) correct identifications and (12) incorrect, and Pylint, which only had (8) correct identifications with (19) incorrect. The remaining LLMs all had relatively poor performances, with both Qwen and ChatGPT having (6) correct identifications and (21)

incorrect. However, the poorest models were Kimi and DeepSeek, which only had (4) correct identifications each and (23) incorrect. Thus, these results demonstrate that some LLMs (like Claude) can indeed be more successful than even traditional static code analysis tools at identifying coding violations within code. Furthermore, these results indicate that there is still much room for improvement for LLMs to be useful in this static code analysis task. Additionally, these results indicate which aspects of the LLMs performance could be improved in order to make those LLMs better suited to performing static code analysis tasks (e.g., reducing the FP while maintaining good detection rates).

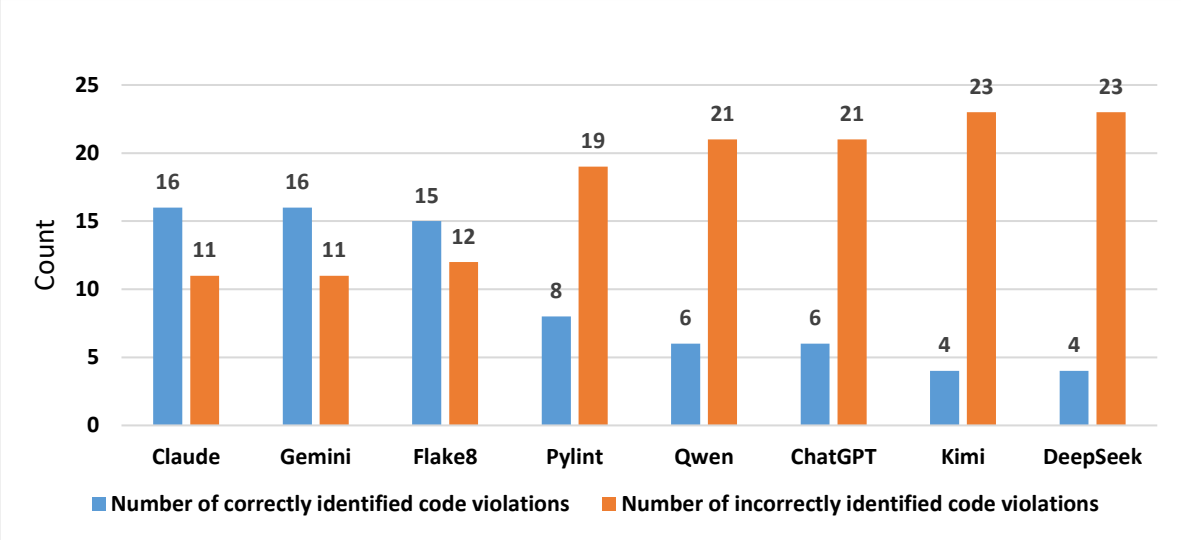

Figure 4.2: Detected violations by LLMs vs. static code analysis tools

Additionally, the capability of LLMs to detect violations that the static code analysis tools missed is shown in Table 4.2, which reveals that LLMs are generally more effective than static code analysis tools, with ChatGPT being the best at finding stylistic violations, and Claude for violations related to documentation. Gemini and Qwen both note similar violations related to documentation, while Kimi highlights violations related to the consistency of style. Lastly, DeepSeek did not provide any additional violations beyond that of the static code analysis tools, indicating their lack of performance compared to each of these other LLMs. Thus, despite the variances in performance, overall, LLMs provided specific areas in which they could be utilized to locate violations that may otherwise be missed by static code analysis tools, especially those related to documentation or even design.

Table 4.2: LLMs' detection of violations missed by static code analysis tools

#	LLMs	Violations	Types
1.	ChatGPT	Import-should-be-outside-top-level	Structural/Layout
		Function-return-not-implemented	Design/Completeness
		Class-attributes-should-be-spaced	Layout/Formatting
2.	Claude	Add-comments-to-complex-logic	Documentation/Clarity
		Type-hints-are-not-defined	Documentation/Clarity
		Method-inside-method-(no-single-responsibility)	Design/Modularity
		Function's-input-not-validated-early	Robustness/Security
		Variables-naming-conventions	Naming
3.	Gemini	Add-comments-to-complex-logic	Documentation/Clarity
		Function's-input-not-validated-early	Robustness/Security
		Type-hints-are-not-defined	Documentation/Clarity
4.	Qwen	Type-hints-are-not-defined	Documentation/Clarity
		Avoid-hardcoding-values-(inside-function-calling)	Naming
5.	Kimi	Use-consistent-string-quotes	Layout/Formatting
6.	DeepSeek	None	

Collectively, Table 4.3 brings together the coding violations that LLMs correctly identify (marked with a ✓ sign) and miss (marked with a × sign), which demonstrate the limitations of LLMs. Claude and Gemini emerge on top, performing best with almost perfect detection for layout and formatting issues like blank lines and line length. Yet, even the leading LLMs struggle to go beyond the surface level. Claude was the only LLM that managed to detect modularity violations like nested methods, although it failed to detect class defined within methods. Gemini, on the other hand, was unique LLM that succeeded in detecting runtime and logical errors within the code, such as undefined variables, all other LLMs failed to do. There was a drop in quality with the performance of ChatGPT, Qwen, Kimi, and DeepSeek. These LLMs failed in most categories, and they performed especially poorly in documentation and naming conventions. Kimi and DeepSeek represented the lowest tier of performance, failing to detect the majority of structural violations or even security violations. Notably, all evaluated LLMs regardless of their general capability proved entirely ineffective at identifying deeper structural issues related to code complexity, maintainability, and refactoring logic. Overall, while LLMs excel at simple formatting and basic stylistic checks, they lack the depth required to function as stand-alone linters. Hybrid approaches that combine static code analysis tools for syntax checks with LLMs for higher-level guidance are recommended to address these limitations.

Table 4.3: Violations that LLMs detect or miss

#	Violations	Types	Claude	Gemini	Qwen	ChatGP	Kimi	DeepSee
1.	Too-many-local-variables-(>15)	Complexity/Modularity	×	×	×	×	×	×
2.	Function-return-not-implemented	Design/Completeness	×	✓	×	✓	×	×
3.	Class-inside-method	Design/Modularity	×	×	×	×	×	×
4.	Method-inside-method-(no-single-responsibility)	Design/Modularity	✓	×	×	×	×	×
5.	Add-meaningful-comments-to-complex-logic	Documentation/Clarity	✓	✓	×	×	×	×
6.	Function-type-hints-Not-define	Documentation/Clarity	✓	×	✓	×	×	×
7.	Class-doc-string-Not-written	Documentation/Stylistic	✓	✓	×	×	×	×
8.	Function-doc-string-Not-written	Documentation/Stylistic	✓	✓	✓	×	✓	✓
9.	Expected-blank-line-after-class	Layout/Formatting	✓	✓	×	✓	×	×
10.	Expected-blank-line	Layout/Formatting	✓	✓	×	×	×	×
11.	Expected-blank-line-before-a-nested-definition	Layout/Formatting	✓	✓	×	×	×	×
12.	Expected-a-blank-line-after-method	Layout/Formatting	✓	✓	×	×	×	×
13.	Line-too-long-(>-79-characters)	Layout/Formatting	✓	✓	×	×	✓	✓
14.	Missing-Space-around-operator	Layout/Formatting	✓	✓	✓	✓	✓	✓
15.	Missing-whitespace-after-(comma)	Layout/Formatting	✓	✓	✓	✓	×	×
16.	Missing-whitespace-after-(colon)	Layout/Formatting	✓	✓	✓	×	×	×
17.	Unused-variable	Maintainability/Efficiency	×	×	×	×	×	×
18.	Function-name-not-snake-case	Naming	×	✓	✓	✓	✓	×
19.	Variables-naming-conventions	Naming	✓	×	×	×	×	×
20.	Class-name-should-be-snake-case	Naming	×	×	×	×	×	×
21.	Consider-using-enumerate-instead-of-iterating	Readability	×	×	×	×	×	×
22.	Unnecessary-"elif"-after-"return"	Readability/Refactoring	×	×	×	×	×	×
23.	Unnecessary-"else"-after-"return""	Readability/Refactoring	×	×	×	×	×	×
24.	Redefining-built-in-'next'	Reliability/Clarity	×	×	×	×	×	×
25.	Functions-input-not-validated-early	Robustness/Security	✓	✓	×	×	×	×
26.	Undefined-variable	Runtime Error/Logic	×	✓	×	×	×	×
27.	Import-should-be-outside-top-level	Structural/Layout	✓	✓	×	✓	×	✓

To further illustrate the limitations of LLMs, the results presented in Table 4.3. A closer look at the code segment below and Table 4.4 shows how Gemini exemplifies these weaknesses. In the combination Sum function, Gemini detected some stylistic issues (e.g., missing whitespace and missing doc-strings) but failed to flag deeper problems such as method nesting, missing input validation, and lack of comments. Taken together, the aggregate evidence from Table 4.3

and this qualitative case confirms a consistent that LLMs struggle with structural issues related to code complexity, maintainability, and refactoring logic.

```
def combinationSum(candidates,target):
    result=[]
    def backtrack(start,path,remaining):
        if remaining==0:
            result.append(path)
            return
        for i in range(start,len(candidates)):
            if candidates[i]>remaining:
                continue
            backtrack(i,path+[candidates[i]],remaining-candidates[i])
    backtrack(0,[],target)
    return result
```

Ground Truth Violations:

- Function name not in snake case.
- Missing whitespace after comma (8 instances).
- Missing type hints (2 instances).
- Missing function doc-string (2 instances).
- Missing class doc-string.
- Missing space around operator (3 instances).
- Missing blank line before nested definition.
- Missing meaningful comments.
- Method defined inside method.
- Function input not validated early.

Table 4.4: Gemini output vs. ground truth

#	Violation	Gemini Result	Evaluation
1	Missing whitespace after comma	Detected 2	**TP (2), FN (6)**
2	Missing space around operator	Detected 1	**TP (1), FN (2)**
3	Missing function doc-string	Detected 2	**TP (2)**
4	Function name not snake case	Missed	**FN**
5	Missing type hints	Missed	**FN (2)**
6	Missing class doc-string	Missed	**FN**
7	Function input not validated early	Missed	**FN**
8	Missing blank line before nested definition	Missed	**FN**
9	Add meaningful comments	Missed	**FN**
10	Method inside method	Missed	**FN**

4.4 Model Consistency (RQ3)

LLMs can produce unreliable and inconsistent responses due to many reasons including probabilistic and lack of deterministic reasoning mechanisms, where they generate different outputs for the same input. Prompt engineering was used to mitigate this issue, particularly given the limitations of free web-based versions without APIs control. Three types of prompts (P1, P2, and P3) were applied iteratively to batches of 25 code segments. Only clear TP and FN results were collected and the result are shown in Figure 4.3. The results reveal that Claude and Gemini achieved the highest average detection rate, with (5) violations each. Claude demonstrated consistent performance across prompts (4, 7, and 5) while Gemini showed a notable peak in P2, detecting (8) violations. ChatGPT followed with a moderate average (4) violations, while DeepSeek, Qwen, and Kimi exhibited significantly lower detection rates averaging (1, 1, and 0) violations, respectively. The results indicates that while some LLMs can identify violations with contextual or semantic complexity, others struggle with simple violations.

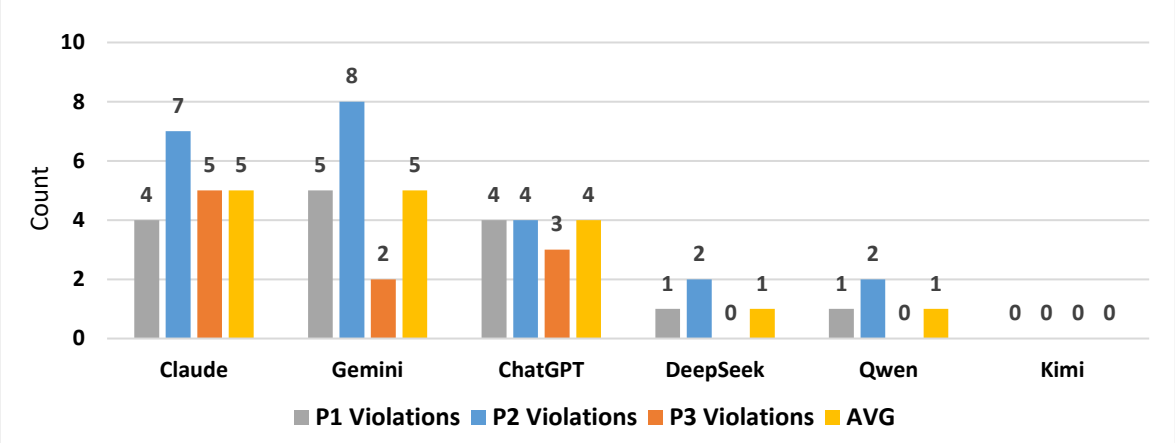

Figure 4.3: Frequency of coding violations with simultaneous TP and FN

To further examine the LLMs for consistency, two additional metrics were utilized: average CR and PA. The results presented in Figure 4.4 showed that the LLMs for reliability were highly inconsistent. Claude had the highest average CR (0.59) and PA (0.41). However, its CR variance (0.70 in P1 vs. 0.50 in P2) still reflects the inconsistencies. Gemini followed with moderate levels of average CR (0.48) and PA (0.32), but it still sees a drastic drop in its CR in P3 (0.24), demonstrating prompt sensitivity. By comparison, ChatGPT has alarmingly low levels of average CR (0.06) and PA (0.16), reflecting the middling detection rates results of violations it previously observed, further demonstrating its unreliability for static code analysis. The other LLMs Qwen, Kimi, and DeepSeek show patterns of complete inconsistency for average CR (for instance, the average CR for DeepSeek was 0.36 for P3 vs. 0.06 for P2) and

show negligible patterns for PA (≤0.13), which correlates with their poor violation detection from the previous findings. Therefore, the results indicate that LLMs even the best-performing LLMs, such as Claude and Gemini, have difficulties employing an LLM for static code analysis, owing to randomness and hallucinations.

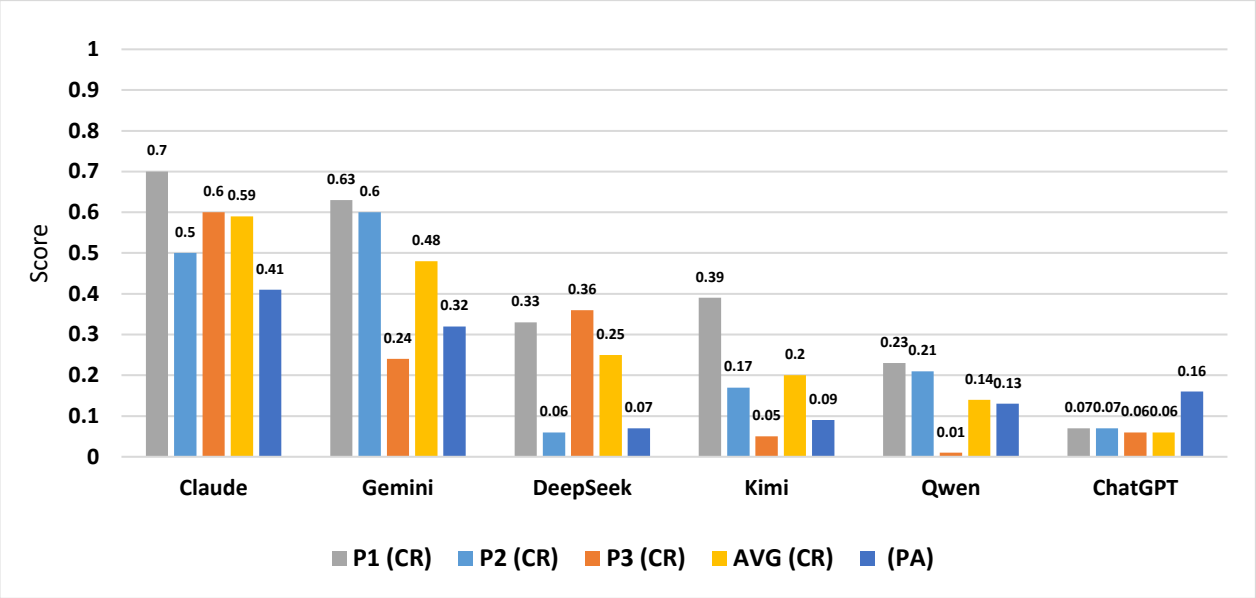

Figure 4.4: Average CR and PA across evaluated LLMs

4.5 Prompting Strategies (RQ4)

Here, three prompt types were tested: P1, P2, and P3, and their performances for the listed LLMs can be seen in Figure 4.5. The results varied. ChatGPT showed excellent Precision (0.99) for P1 and P2 but demonstrated extremely low Recall (0.07) and F1-Scores (0.13), indicating a trade-off between Precision and Recall. Gemini performed notably well with P1, achieving an F1-Score of (0.78), but its performance dropped with P2 (0.72) and P3 (0.39), suggesting that the model is likely best with prompt type P1. Claude had a strong performance for all prompts, showing balanced and high F1-Scores for P1 (0.81) and P3 (0.78), making it a adaptable model. DeepSeek and Qwen showed varied results, while Qwen also had a notably poor performance with prompt type P3, scoring an F1-Score of (0.01). Kimi showed perfect Precision (1.0) for all prompt types, but its Recall and F1-Scores were low, suggesting that the performance may be unbalanced for response generation. Therefore, the results suggest that prompting can affect how well an LLM performs, with the best results for most models for P1.

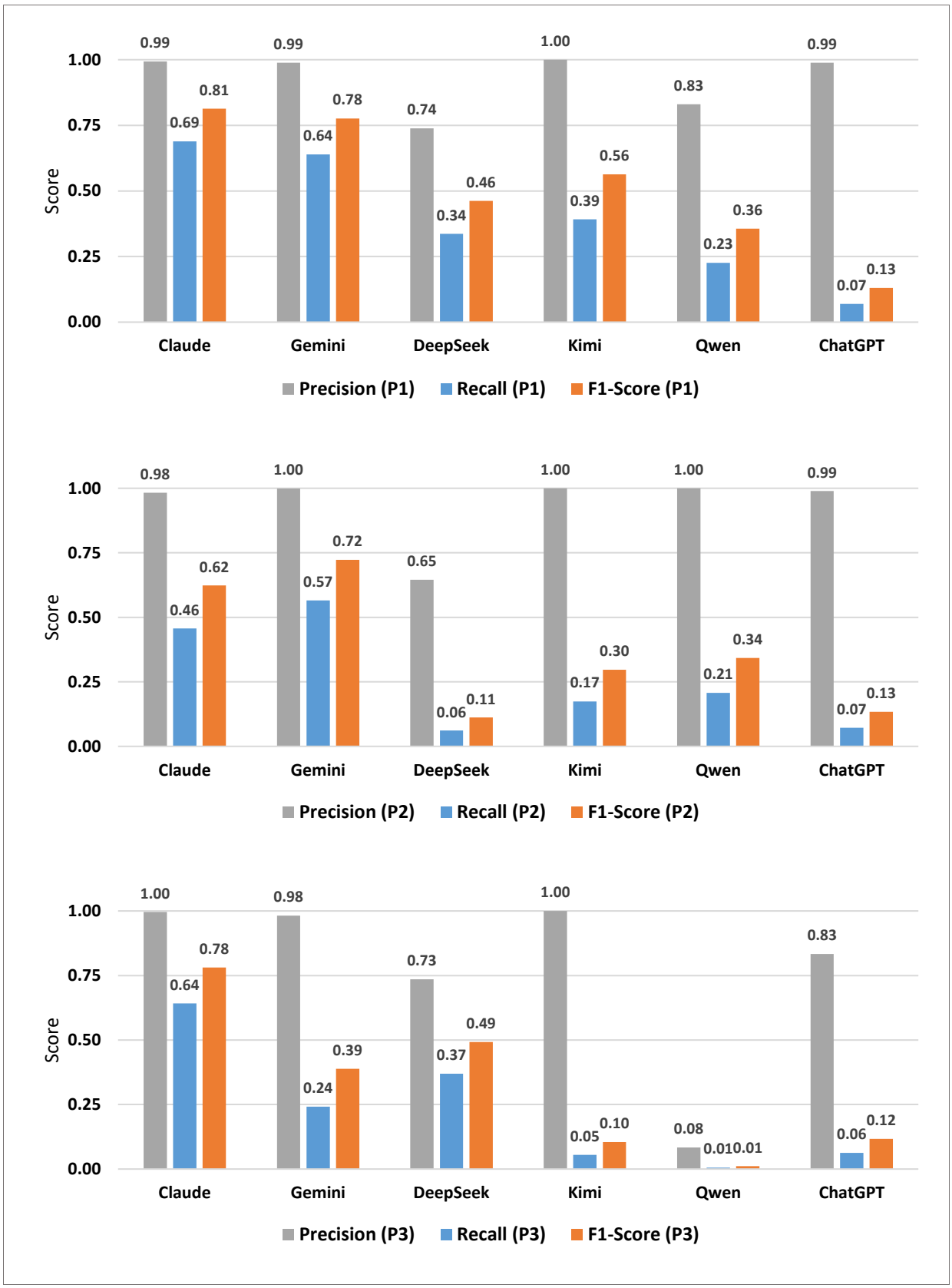

Figure 4.5: Comparing prompt types efficiency in LLMs

4.6 Batch vs. Individual Code Analysis (RQ5)

The effect of processing the code segments in batch or individually, can lead to different numbers of detecting coding violations by the LLMs. Figure 4.6 shows LLM's performance in analyzing the individual code segments by using only base prompt (P1) which shows the best performance among others, as it was discussed earlier. Surprisingly, all the models had perfect Precision score (1.0), which indicates that all the models were correct in their reported violations. Yet, the Recall scores varied largely among the LLMs, which means the models differed in reporting all the existing violations in the code segment. Gemini demonstrated the highest Recall (0.85) and F1-Score (0.92), suggesting robust overall performance, followed by Claude with Recall (0.73) F1-Score (0.85) and ChatGPT with Recall (0.63) F1-Score (0.77) the two models show moderate performance. In contrast, DeepSeek, Qwen, and Kimi showed markedly lower Recall (0.58, 0.44, and 0.42) respectively and F1-Scores, underscoring their limitations in comprehensive violation detection. The findings suggest that while all models excel at avoiding Precision, their utility for thorough static code analysis hinges on improving Recall, particularly for lower-performing models.

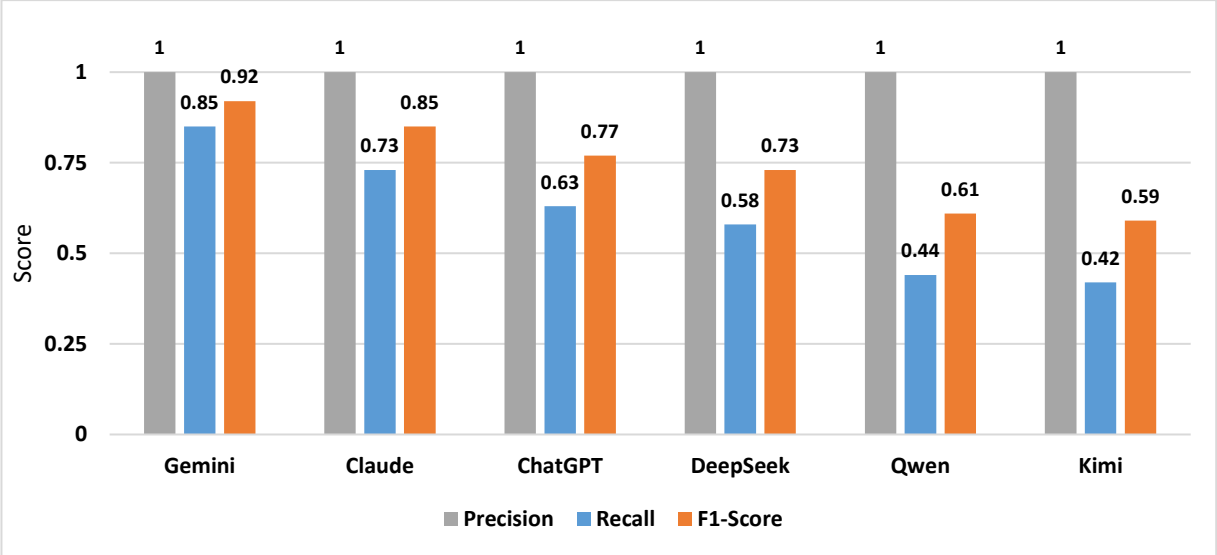

Figure 4.6: Overall performance of LLMs (Individual code)

In addition, Table 4.5 compares the F1-Score performance of the used LLMs when inputting code segments in batch or individually. The results reveal that all LLMs show better performance when code segments are analyzed individually instead of in batch. ChatGPT showed the most significant improvement, from (0.13) in batch performance analysis to (0.77) from individual analysis, with an increase of (+0.64). DeepSeek and Qwen also demonstrated similar performances with increases of (+0.27) and (+0.25), respectively. Gemini had a relatively lower increase of (+0.14), while Claude and Kimi showed the lowest increases with

minimal increases of (+0.04) and (+0.03), respectively. Thus, these results suggest that the individual analysis of codes has a significant beneficial effect over that of batch analysis in terms of the effect on the performance of the LLMs in detecting coding violations.

Table 4.5: F1-Score performance comparison (batch vs. individual code)

#	LLMs	F1-Score (Batch)	F1-Score (Individual)	Overall improvement
1.	ChatGPT	0.13	0.77	+0.64
2.	DeepSeek	0.46	0.73	+0.27
3.	Qwen	0.36	0.61	+0.25
4.	Gemini	0.78	0.92	+0.14
5.	Claude	0.81	0.85	+0.04
6.	Kimi	0.56	0.59	+0.03

To sum up, the findings show that LLM performance varies widely, with Claude and Gemini consistently outperforming other models and, in some cases, rivaling or exceeding traditional static analysis tools such as Flake8, while Pylint and several LLMs suffer from very low Recall. Although many LLMs achieve high Precision, their reliability is limited by missed violations, prompt sensitivity, and inconsistent outputs. LLMs demonstrate strengths in detecting documentation, stylistic, and certain modularity violations that traditional tools miss, regardless of their general capability proved entirely ineffective at identifying deeper structural issues related to code complexity, maintainability, and refactoring logic. Prompt design significantly affects performance, with structured prompts generally yielding the most stable results. Additionally, analyzing code segments individually leads to notable performance improvements compared to batch processing, particularly for weaker models. Overall, the results confirm that LLMs are not yet reliable as standalone static analyzers but can effectively complement traditional tools, supporting hybrid approaches for more comprehensive Python code quality assessment.

Chapter Five

Conclusions and Future Works

Chapter 5: Conclusions and Future Works

5.1 Conclusions

In this thesis, the ability of LLMs in detecting Python coding violations was evaluated, and their effectiveness was contrasted with traditional Python static code analysis tools. The results show that while LLMs like Claude Sonnet and Gemini may compete with or even outperform static code analysis tools in some cases, their efficacy varies considerably depending on the model and type of violation. Overall, Claude Sonnet emerged as the top-performing LLM (F1-Score: 0.81), showcasing strengths in contextual understanding and Recall, while static code analysis tools like Flake8 excelled in Precision (1.00). In terms of advantages and disadvantages, within the context of Python, LLMs can complement static code analysis tools, but their non-deterministic nature and susceptibility to false negatives make them unsuitable as standalone tools for security-critical applications. Regarding the prompting strategies, structural prompts (P1) yielded the most balanced results, whereas CoT and role-based prompts showed mixed efficacy, underscoring the importance of prompt engineering. LLMs faced limitations in batch processing and long-context analysis, with individual code segments evaluation proving more reliable. The contributions of this thesis lie in the empirical evidence and practical insights for utilizing LLMs for static code analysis.

5.2 Future Works

The work of this thesis can be extended by including more LLMs (including reasoning-capable LLMs), additional prompt types, other programming languages, and a wider range of coding violations. Moreover, studying combining or selecting the best result of different LLMs with static code analysis tools to enhance the overall results. Finally, developing a larger Python benchmark dataset for static code analysis would support researchers in contributing effectively to this rapidly evolving field.

References

[1] N. Wadhwa *et al.*, “CORE: Resolving Code Quality Issues using LLMs,” *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 789–811, Jul. 2024, doi: 10.1145/3643762.

[2] K. Moratis, T. Diamantopoulos, D. N. Nastos, and A. Symeonidis, “Write me this Code: An Analysis of ChatGPT Quality for Producing Source Code,” *Proc. - 2024 IEEE/ACM 21st Int. Conf. Min. Softw. Repos. MSR 2024*, pp. 147–151, 2024, doi: 10.1145/3643991.3645070.

[3] E. A. AlOmar and M. W. Mkaouer, “Cultivating Software Quality Improvement in the Classroom: An Experience with ChatGPT,” in *2024 36th International Conference on Software Engineering Education and Training (CSEE&T)*, IEEE, Jul. 2024, pp. 1–10. doi: 10.1109/CSEET62301.2024.10663028.

[4] Z. Guo *et al.*, “Mitigating False Positive Static Analysis Warnings: Progress, Challenges, and Opportunities,” *IEEE Trans. Softw. Eng.*, vol. 49, no. 12, pp. 5154–5188, Dec. 2023, doi: 10.1109/TSE.2023.3329667.

[5] M. M. Mohajer *et al.*, “SkipAnalyzer: A Tool for Static Code Analysis with Large Language Models,” Dec. 2023, doi: 10.48550/arXiv.2310.18532.

[6] H. Li, Y. Hao, Y. Zhai, and Z. Qian, “Assisting Static Analysis with Large Language Models: A ChatGPT Experiment,” in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, New York, NY, USA: ACM, Nov. 2023, pp. 2107–2111. doi: 10.1145/3611643.3613078.

[7] Q. Zhang *et al.*, “A Survey on Large Language Models for Software Engineering,” Sep. 2024, doi: 10.48550/arXiv.2312.15223.

[8] Z. Ságodi, I. Siket, and R. Ferenc, “Methodology for Code Synthesis Evaluation of LLMs Presented by a Case Study of ChatGPT and Copilot,” *IEEE Access*, vol. 12, pp. 72303–72316, 2024, doi: 10.1109/ACCESS.2024.3403858.

[9] M. Das Purba, A. Ghosh, B. J. Radford, and B. Chu, “Software Vulnerability Detection using Large Language Models,” in *2023 IEEE 34th International Symposium on Software Reliability Engineering Workshops (ISSREW)*, IEEE, Oct. 2023, pp. 112–119. doi: 10.1109/ISSREW60843.2023.00058.

[10] W. Ma *et al.*, “LLMs: Understanding Code Syntax and Semantics for Code Analysis,” Feb. 2024, doi: 10.48550/arXiv.2305.12138.

[11] P. Haindl and G. Weinberger, “Does ChatGPT Help Novice Programmers Write Better Code? Results From Static Code Analysis,” *IEEE Access*, vol. 12, no. July, pp. 114146–114156, 2024, doi: 10.1109/ACCESS.2024.3445432.

[12] K. Jesse, T. Ahmed, P. T. Devanbu, and E. Morgan, “Large Language Models and Simple, Stupid Bugs,” in *2023 IEEE/ACM 20th International Conference on Mining Software Repositories (MSR)*, IEEE, May 2023, pp. 563–575. doi: 10.1109/MSR59073.2023.00082.

[13] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, “Examining Zero-Shot Vulnerability Repair with Large Language Models,” in *2023 IEEE Symposium on Security and Privacy (SP)*, IEEE, May 2023, pp. 2339–2356. doi: 10.1109/SP46215.2023.10179420.

[14] S. Raschka, J. Patterson, and C. Nolet, “Machine Learning in Python: Main developments and technology trends in data science, machine learning, and artificial intelligence,” *Inf.*, vol. 11, no. 4, Feb. 2020, doi: 10.3390/info11040193.

[15] M. Butwall, P. Ranka, and S. Shah, “Python in Field of Data Science: A Review,” *Int. J. Comput. Appl.*, vol. 178, no. 49, pp. 20–24, Sep. 2019, doi: 10.5120/ijca2019919404.

[16] S. R. Danilo Nikolić, Darko Stefanović, Dušanka Dakić, Srđan Sladojević, “Analysis of the Tools for Static Code Analysis.” Conference: 2021 20th International Symposium INFOTEH-JAHORINA (INFOTEH), 2021. doi: 10.1109/INFOTEH51037.2021.9400688.

[17] V. Dimaridou, A. Kyprianidis, and M. Papamichail, “Towards Modeling the User-Perceived Quality of Source Code using Static Analysis Metrics,” *Int. Conf. Softw. Technol. (ICSOFT 2017)*, no. 12, pp. 73–84, 2017, doi: 10.5220/0006420000730084.

[18] P. Louridas, “Static code analysis,” *IEEE Softw.*, vol. 23, no. 4, pp. 58–61, 2006, doi: 10.1109/MS.2006.114.

[19] Z. Zheng *et al.*, “Towards an Understanding of Large Language Models in Software Engineering Tasks,” *Empir. Softw. Eng.*, vol. 30, no. 2, p. 50, Aug. 2023, doi: 10.1007/s10664-024-10602-0.

[20] J. Gong *et al.*, “Language Models for Code Optimization: Survey, Challenges and Future Directions,” Jan. 2025, doi: 10.48550/arXiv:2501.01277.

[21] Z. Zheng *et al.*, “A Survey of Large Language Models for Code: Evolution, Benchmarking, and Future Trends,” Nov. 2024, doi: 10.48550/arXiv:2311.10372.

[22] T.-T. Nguyen, T. T. Vu, H. D. Vo, and S. Nguyen, “An empirical study on capability of Large Language Models in understanding code semantics,” *Inf. Softw. Technol.*, vol. 185, p. 107780, Sep. 2025, doi: 10.1016/j.infsof.2025.107780.

[23] L. Ardito, M. Ballario, and M. Valsesia, “Research, Implementation and Analysis of Source Code Metrics in Rust-Code-Analysis,” in *2023 IEEE 23rd International Conference on Software Quality, Reliability, and Security (QRS)*, IEEE, Oct. 2023, pp. 497–506. doi: 10.1109/QRS60937.2023.00055.

[24] G. Morgachev, V. Ignatyev, and A. Belevantsev, “Detection of variable misuse using static analysis combined with machine learning,” *Proc. - 2019 Ivannikov Ispras Open Conf. ISPRAS 2019*, pp. 16–24, 2019, doi: 10.1109/ISPRAS47671.2019.00009.

[25] V. N. Ignatyev, N. V. Shimchik, D. D. Panov, and A. A. Mitrofanov, “Large language models in source code static analysis,” in *2024 Ivannikov Memorial Workshop (IVMEM)*, IEEE, May 2024, pp. 28–35. doi: 10.1109/IVMEM63006.2024.10659715.

[26] A. Amburle, C. Almeida, N. Lopes, and O. Lopes, “AI based Code Error Explainer using Gemini Model,” in *2024 3rd International Conference on Applied Artificial Intelligence and Computing (ICAAIC)*, IEEE, Jun. 2024, pp. 274–278. doi: 10.1109/ICAAIC60222.2024.10574931.

[27] J. Ramamoorthy, K. Gupta, R. C. Kafle, N. K. Shashidhar, and C. Varol, “A Novel Static Analysis Approach Using System Calls for Linux IoT Malware Detection,” *Electronics*, vol. 13, no. 15, p. 2906, Jul. 2024, doi: 10.3390/electronics13152906.

[28] H. B. Hassan, Q. I. Sarhan, and Á. Beszédes, “Evaluating Python Static Code Analysis Tools Using FAIR Principles,” *IEEE Access*, vol. 12, no. September, pp. 173647–

173659, 2024, doi: 10.1109/ACCESS.2024.3503493.

[29] I. Kotenko, K. Izrailov, and M. Buinevich, “Static Analysis of Information Systems for IoT Cyber Security: A Survey of Machine Learning Approaches,” *Sensors*, vol. 22, no. 4, p. 1335, Feb. 2022, doi: 10.3390/s22041335.

[30] C. Fang *et al.*, “Large Language Models for Code Analysis: Do LLMs Really Do Their Job?,” Oct. 2023, doi: 10.48550/arXiv:2310.12357.

[31] R. Bairi *et al.*, “CodePlan: Repository-Level Coding using LLMs and Planning,” *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 675–698, Jul. 2024, doi: 10.1145/3643757.

[32] H. Li, Y. Hao, Y. Zhai, and Z. Qian, “The Hitchhiker’s Guide to Program Analysis: A Journey with Large Language Models,” no. 1, Nov. 2023, doi: 10.48550/arXiv.2308.00245.

[33] S. Sikand, R. Mehra, V. S. Sharma, V. Kaulgud, S. Podder, and A. P. Burden, “Do Generative AI Tools Ensure Green Code? An Investigative Study,” in *Proceedings of the 2nd International Workshop on Responsible AI Engineering*, New York, NY, USA: ACM, Apr. 2024, pp. 52–55. doi: 10.1145/3643691.3648588.

[34] K. Petersen, S. Vakkalanka, and L. Kuzniarz, “Guidelines for conducting systematic mapping studies in software engineering: An update,” *Inf. Softw. Technol.*, vol. 64, no. 5, pp. 1–18, Aug. 2015, doi: 10.1016/j.infsof.2015.03.007.

[35] P. Brereton, B. A. Kitchenham, D. Budgen, M. Turner, and M. Khalil, “Lessons from applying the systematic literature review process within the software engineering domain,” *J. Syst. Softw.*, vol. 80, no. 4, pp. 571–583, Apr. 2007, doi: 10.1016/j.jss.2006.07.009.

[36] C. Wohlin, “Guidelines for snowballing in systematic literature studies and a replication in software engineering,” *ACM Int. Conf. Proceeding Ser.*, 2014, doi: 10.1145/2601248.2601268.

[37] W. Rahmaniar, “ChatGPT for Software Development: Opportunities and Challenges,” *IT Prof.*, vol. 26, no. 3, pp. 80–86, May 2024, doi: 10.1109/MITP.2024.3379831.

[38] H. Hajipour, K. Hassler, T. Holz, L. Schönherr, and M. Fritz, “CodeLMSec Benchmark: Systematically Evaluating and Finding Security Vulnerabilities in Black-Box Code Language Models,” in *2024 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*, IEEE, Apr. 2024, pp. 684–709. doi: 10.1109/SaTML59370.2024.00040.

[39] Z. Yuan *et al.*, “Evaluating and Improving ChatGPT for Unit Test Generation,” *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 1703–1726, 2024, doi: 10.1145/3660783.

[40] A. A. Mahyari, “Harnessing the Power of LLMs in Source Code Vulnerability Detection,” in *MILCOM 2024 - 2024 IEEE Military Communications Conference (MILCOM)*, IEEE, Oct. 2024, pp. 251–256. doi: 10.1109/MILCOM61039.2024.10774025.

[41] N. S. Mathews, Y. Brus, Y. Aafer, M. Nagappan, and S. McIntosh, “LLbezpeky: Leveraging Large Language Models for Vulnerability Detection,” Jan. 2024, doi: 10.48550/arXiv:2401.01269.

[42] H. Li and L. Shan, “LLM-based Vulnerability Detection,” in *2023 International Conference on Human-Centered Cognitive Systems (HCCS)*, IEEE, Dec. 2023, pp. 1–4. doi: 10.1109/HCCS59561.2023.10452613.

[43] A. A. Hossain, M. K. PK, J. Zhang, and F. Amsaad, “Malicious Code Detection Using LLM,” in *NAECON 2024 - IEEE National Aerospace and Electronics Conference*, IEEE, Jul. 2024, pp. 414–416. doi: 10.1109/NAECON61878.2024.10670668.

[44] J. M. Gonzalez-Barahona, “Software development in the age of LLMs and XR,” in *Proceedings of the 1st ACM/IEEE Workshop on Integrated Development Environments*, New York, NY, USA: ACM, Apr. 2024, pp. 66–69. doi: 10.1145/3643796.3648457.

[45] M. Omar and S. Shiaeles, “VulDetect: A novel technique for detecting software vulnerabilities using Language Models,” in *2023 IEEE International Conference on Cyber Security and Resilience (CSR)*, IEEE, Jul. 2023, pp. 105–110. doi: 10.1109/CSR57506.2023.10224924.

[46] Q. Guo *et al.*, “Exploring the Potential of ChatGPT in Automated Code Refinement: An Empirical Study,” in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, New York, NY, USA: ACM, Feb. 2024, pp. 1–13. doi: 10.1145/3597503.3623306.

[47] A. P. S. Venkatesh, S. Sabu, A. M. Mir, S. Reis, and E. Bodden, “The Emergence of Large Language Models in Static Analysis: A First Look through Micro-Benchmarks,” in *Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering*, New York, NY, USA: ACM, Apr. 2024, pp. 35–39. doi: 10.1145/3650105.3652288.

[48] N. Souma *et al.*, “Can ChatGPT Correct Code Based on Logical Steps,” *Proc. - Asia-Pacific Softw. Eng. Conf. APSEC*, no. Ii, pp. 653–654, 2023, doi: 10.1109/APSEC60848.2023.00094.

[49] Y. Bajpai *et al.*, “Let’s Fix this Together: Conversational Debugging with GitHub Copilot,” in *2024 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*, IEEE, Sep. 2024, pp. 1–12. doi: 10.1109/VL/HCC60511.2024.00011.

[50] X. Yin, C. Ni, and S. Wang, “Multitask-based Evaluation of Open-Source LLM on Software Vulnerability,” *IEEE Trans. Softw. Eng.*, vol. 50, no. 11, pp. 3071–3087, Apr. 2024, doi: 10.1109/TSE.2024.3470333.

[51] V. Akuthota, R. Kasula, S. T. Sumona, M. Mohiuddin, M. T. Reza, and M. M. Rahman, “Vulnerability Detection and Monitoring Using LLM,” *Proc. 2023 IEEE 9th Int. Women Eng. Conf. Electr. Comput. Eng. WIECON-ECE 2023*, pp. 309–314, 2023, doi: 10.1109/WIECON-ECE60392.2023.10456393.

[52] N. K. Gupta, A. Chaudhary, R. Singh, and R. Singh, “ChatGPT: Exploring the Capabilities and Limitations of a Large Language Model for Conversational AI,” in *2023 International Conference on Advances in Computation, Communication and Information Technology (ICAICCIT)*, IEEE, Nov. 2023, pp. 139–142. doi: 10.1109/ICAICCIT60255.2023.10465811.

[53] Z. Liu, Z. Yang, and Q. Liao, “Exploration On Prompting LLM With Code-Specific Information For Vulnerability Detection,” *Proc. - 2024 IEEE Int. Conf. Softw. Serv. Eng. SSE 2024*, pp. 273–281, 2024, doi: 10.1109/SSE62657.2024.00049.

[54] J. Villmow, V. Campos, J. Petry, A. Abbad-Andaloussi, A. Ulges, and B. Weber, “How Well Can Masked Language Models Spot Identifiers That Violate Naming Guidelines?,” in *2023 IEEE 23rd International Working Conference on Source Code*

Analysis and Manipulation (SCAM), IEEE, Oct. 2023, pp. 131–142. doi: 10.1109/SCAM59687.2023.00023.

[55] P. Di *et al.*, "CodeFuse-13B: A Pretrained Multi-lingual Code Large Language Model," in *Proceedings of the 46th International Conference on Software Engineering: Software Engineering in Practice*, New York, NY, USA: ACM, Apr. 2024, pp. 418–429. doi: 10.1145/3639477.3639719.

[56] D. Noever, "Can Large Language Models Find And Fix Vulnerable Software?," Aug. 2023, doi: 10.48550/arXiv.2308.10345.

[57] Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities," *Int. Conf. Learn. Represent.*, pp. 1–24, Apr. 2025, doi: 10.48550/arXiv.2405.17238.

[58] Pylint-Dev., "Pylint: It's Not Just a Linter That Annoys You." 2024. Accessed: Feb. 17, 2025. [Online]. Available: https://github.com/pylint-dev/pylint

[59] Flake8-Dev, "Flake8 : is a python tool that check the style and quality of some python code." 2025. Accessed: Feb. 17, 2025. [Online]. Available: https://github.com/PyCQA/flake8

[60] A. Birillo *et al.*, *One Step at a Time: Combining LLMs and Static Analysis to Generate Next-Step Hints for Programming Tasks*, no. 9. Koli Calling '24: Proceedings of the 24th Koli Calling International Conference on Computing Education Research, 2024. doi: 10.1145/3699538.3699556.

[61] K. A. M. Carandang *et al.*, "Are LLMs reliable? An exploration of the reliability of large language models in clinical note generation," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 6: Industry Track)*, Stroudsburg, PA, USA: Association for Computational Linguistics, May 2025, pp. 1413–1422. doi: 10.18653/v1/2025.acl-industry.99.

[62] S. M. Mousavi, S. Alghisi, and G. Riccardi, "LLMs as Repositories of Factual Knowledge: Limitations and Solutions," Jan. 2025, doi: 10.48550/arXiv.2501.12774.

Appendix A: Confusion Matrix–Based Performance Comparison

Table 1: Performance comparison of LLMS across different prompting strategies

Model	Prompt	TP	FP	FN	Precision	Recall	F1-Score
ChatGPT 4-Turbo	P1	96	1	1273	0.99	0.07	0.13
	P2	96	1	1273	0.99	0.07	0.13
	P3	84	17	1269	0.83	0.06	0.12
Gemini 2.5-pro-preview-	P1	871	9	490	0.99	0.64	0.78
03-25c	P2	778	0	592	1.00	0.57	0.72
	P3	332	7	1031	0.98	0.24	0.39
Claude 3.7 Sonnet	P1	934	9	427	0.99	0.69	0.81
	P2	620	12	738	0.98	0.46	0.62
	P3	871	0	499	1.00	0.64	0.78
DeepSeek-V3	P1	413	145	812	0.74	0.34	0.46
	P2	80	43	1247	0.65	0.06	0.11
	P3	447	165	758	0.73	0.37	0.49
Qwen 2.5-Max	P1	301	61	1008	0.83	0.23	0.36
	P2	282	0	1088	1.00	0.21	0.34
	P3	10	115	1245	0.08	0.01	0.01
Kimi	P1	534	0	836	1.00	0.39	0.56
	P2	239	0	1131	1.00	0.17	0.30
	P3	69	0	1301	1.00	0.05	0.10

Table 2: Performance comparison of traditional static analysis tools

Static Analysis Tool	TP	FP	FN	Precision	Recall	F1-Score
Pylint	288	0	1082	1.00	0.21	0.35
Flake8	903	0	467	1.00	0.66	0.79

حکومەتا هەرێما کوردستانا عیراقێ
وەزارەتا خواندنا بلند و ڤەکۆلینێن زانستی
زانکویا دهۆك
کولیژا زانست
پشکا زانستێن کومپیوتەری

هەلسەنگاندنا مودێلێن مەزن یێن زمانێ (LLMs) ژبو شروڤەکرنا ستاتیک یا کودی

نامەیەکا پێشکێشکریە بۆ جڤاتا کولیژا زانست ل زانکۆیا دهۆک وەک پشکەک ژ پێداویستیێن بدەستڤائینانا پلا ماستەر ب زانستێن کومپیوتەری

یا قوتابی
هکار ازور محمدصالح
بەکالوریوس ب زانستێن کومپیوتەری، زانکویا نەوروز – 2014

سەرپەرشت
پ. هـ. د. قصی ادریس سرحان
زانکویا دهوک، دهوک، هەرێما کوردستانێ، عیراق

دهۆک – هەرێما کوردستانێ - عیراق

2726 کوردی **2026 زاینی** **1447 کۆچی**

پەیرەوکرنا پراکتیکێن درست یێن کۆدینکێ گرنگە ژبۆ باشترکرن و خواندنا بساناهی یا کودێ نڤیسای، هەروەسا ژبو پاراستن و پشترِاستیا کودی. ئامێرێن شروڤەکرنا کۆدێ ستاتیکی یێن ئاسایی کو کارێ ڤەدیتنا سەرپێچیێن کۆدینکێ دکەن بێکو کۆدێ بجهبینن و بدەن کاری ، وەکی Pylint و Flake8 ، کو ئەڤ ئامیرە ب شێوەکێ بەرفرەھـ بکاردێن ژبۆ سەپاندنا کوالیتیا بلند یا کۆدێ پایتونی. ئەڤان ئامیران توانایێن سنوردار ھەنە ژبۆ پێشکێشکرنا تێگەھشتنا کۆدینکا درست و لۆژیکیا کۆنتێکستی. ئەڤ تیزە کارێ ڤەکولینا مۆدێلێن زمانێ یێن مەزن (LLMs) بەراوردکەت دگەل ئامێرێن ڤەکولینا کۆدێ ستاتیکی د ڤەدیتنا سەرپێچیێن کۆدینکێ د زمانێ پایتونی دا. شەش LLM یێن مودرێن: ChatGPT ، Gemini، Claude Sonnet، DeepSeek، Kimi، و Qwen ھاتینە ھەلسەنگاندن بەرامبەر دوو ئامێرێن کۆدێ ستاتیک Pylint وFlake8 . ژبۆ ڤی کاری داتاسێتا ھاتیە بکارئینان ھاتیە ئامادەکرن ژ ٧٥ پارچێن کۆدێ پایتونی، و ٢٧ سەرپێچیێن کۆدینکێ کو بدەست ھاتینە نیشانکرن. ھەروەسا، سێ ستراتیژیێن پرۆمپتکرنێ: بنیاتدانای، زنجیرەیا رامانی، و شێوازێ ئەرک، ھاتنە بکارئێنان بۆ ڕێنماییکرنا LLM یێن ھەلبژاردی.

ئەنجامێن ڤەکولینێ ئاشکرا دکەن کو Claude Sonnet بلندترین F1-Score بدەستڤەئینا (0.81)، کو باشترە و ئاست بەرزترە ژ Flake8 (0.79) و Gemini (0.78). Claude نیشان دا کو ھویربینەکا (Precision) بھێز ھەیە (0.99) و بیرئینانەکا (Recall) یا باش (0.69) ھەیە. لێ، LLM ێن دی جیاوازی دپێشکێشکرنا کاری خودا نیشان دایە، کو Kimi ، DeepSeek، Qwen و ChatGPT ئەنجامێن وان کێمترن ژ یێن دی تر. ب تایبەتی، Pylint و ChatGPT لاوازترین ئەنجام پێشکێشکرن ، ب F1-Score ێن (0.35) و (0.13) لدویف ئێک، کو ئەگەرێ وان ئەنجامان بیرئینانەڤەیا زۆر نزم (0.21) و (0.07) بو سەرەڕای ھویربینیا تەمام یان نزیکی تەمام بوونێ. ھەروەسا کارێ ڤەکولینا ئەنجامان دیار کریە کو KIMI جھێ باوەریێ یە د (False Discovery Rate) دا کو ئەنجامێ (0.00) بدەست ئینایە, ل لایەکێ دی Qwen نە بویە جھێ بدەستڤەئینانا ئەنجامێن جێگیر کو (0.92) بدەستخوڤە ئینایە ل ئێک ژ پرومپتان. و لسەر ئاستێ داتایێن جوری LLM ـــا شیانێن تایبەت ھەنە د دیار کرنا خەلەتیێن کودی یێن کومێنت دانانێ و خەلەتیێن گرێدای ستایلێ کودی یێن ئامیرێن Pylint و Flake8 نەشیای دەست نیشان بکەت, لێ LLM نەشیاینە خەلەتیێن بنیاتێ کودی یێن ئالوز دەست نیشان بکەن سەرەڕای ھویربینەکا تەمام یان یا نزیکی تەمام بونێ. ئەنجامێن تیزێ ب شێوەکێ مەزن ژلایێ رێکا پرۆمپتکرنێ کاریگەری ھەبوویە، کە پرۆمپتا بنیاتدانای د زۆربەیا کەیسان دا پێشکێشکرنەکا ھاوسەنگتر نیشان دایە. ھەروەسا ئەنجامان دیار کریە کو ھەژمارا کودێن ھاتینە پێدان بو LLM ـا دگەل پرومپتێ کاریگەری ھەبویە کو لدەمێ کود ب شێوێ تاکانە ھاتیە پێدان ئەنجام بشێوەیەکێ گشتی باشتر بوون بتایبەتی ل ChatGPT کو (+0.64) F1-Score زێدەبویە. ئەڤ تیزە بەشدارە دکارێ زەنگینکرنا بکارئێنانا LLM ان بۆ پاراستنا کوالیتیا کۆدی، ھەروەسا رۆلێ وان یێ پێشبینیکری وەک تەمامکەرێن ئامێرێن ڤەکولینا کۆدێ ستاتیک بۆ زمانێ پایتونی نیشان ددەت.

پەیڤێن کلیل:

مودێلێن زمانی یێن مەزن, شروڤەکرنا ستاتیک یا کۆدی, کوالیتیا کودی, خەلەتیێن کودی, پایثون

حكومة إقليم كوردستان – العراق

وزارة التعليم العالي والبحث العلمي

جامعة دهوك

كلية العلوم

قسم علوم الحاسبات

تقييم نماذج اللغة الكبيرة (LLMs) للتحليل الساكن للكود

إطروحة ماجستير

مقدمة الى مجلس كلية العلوم في جامعة دهوك إستكمالا لمتطلبات الحصول على درجة ماجستير في علوم الحاسيبات

من قبل الباحث

هكار ازور محمد صالح

بكالوريوس علوم الحاسبات، جامعة نوروز – 2014

بإشراف

أ. م. د. قصى ادريس سرحان

جامعة دهوك، دهوك، إقليم كوردستان، جمهورية العراق

دهوك، إقليم كوردستان، جمهورية العراق

٢٧٢٦ ك **٢٠٢٦ م** **١٤٤٧ هـ**

الخلاصة:

الالتزام بممارسات البرمجة الجيدة أمرٌ بالغ الأهمية لتعزيز قابلية قراءة البرمجيات وسهولة صيانتها وموثوقيتها. تُستخدم أدوات التحليل الساكن الشائعة للغة Python ، مثل Pylint و Flake8، على نطاق واسع لفرض جودة الاكواد البرمجية من خلال اكتشاف مخالفات الترميز دون تنفيذ الكود، إلا أنها تعاني من قدرات محدودة في توفير فهم عميق الكود والاستدلال السياقي. تبحث هذه الرسالة في فعالية نماذج اللغات الكبيرة (LLMs) مقارنةً بأدوات التحليل الساكن التقليدية في اكتشاف مخالفات الكود Python . تم تقييم ستة نماذج حديثة من نماذج اللغات الكبيرة، وهي: ChatGPT وGemini وClaude Sonnet وDeepSeek وKimi وQwen ، ومقارنتها بأداتي Pylint و Flake8 . ولتحقيق ذلك، تم استخدام مجموعة بيانات منسقة تتكون من 75 مقطع الكود Python، وموسومة يدويًا بـ 27 نوعًا شائعًا من مخالفات الكود. بالإضافة إلى ذلك، تم اعتماد ثلاث استراتيجيات تلقين شائعة لتوجيه النماذج المختارة، وهي: التلقين البنيوي، وسلسلة التفكير، والتلقين القائم على الدور.

أظهرت النتائج التجريبية أن أفضل النماذج أداءً، وهو Claude Sonnet، حقق أعلى قيمة لمقياس F1-Score بلغت (0.81)، متفوقًا على Flake8 الذي سجل (0.79)، وعلى Gemini الذي سجل (0.78) . كما حقق Claude قيمًا مرتفعة لكل من الدقة (Precision) البالغة (0.99) والاسترجاع (Recall) البالغ (0.69)، في حين أظهرت بقية نماذج اللغات الكبيرة نتائج متباينة، حيث جاء أداء Kimi وDeepSeek وQwen وChatGPT أقل مقارنةً بالنماذج الأخرى. وأظهر كل من Pylint وChatGPT أضعف أداء، بقيم F1-Score بلغت (0.35) و (0.13) على التوالي، ويُعزى ذلك أساسًا إلى انخفاض معدل الاسترجاع لديهما بشكل كبير، إذ بلغ (0.21) و (0.07) . كما كشف تحليل النتائج عن وجود تباين مرتفع في الموثوقية، حيث حقق Kimi معدل اكتشاف خاطئ (False Discovery Rate) مثاليًا بلغ (0.00) ، في حين أظهر Qwen تقلبًا شديدًا في قياساته، إذ وصل معدل الاكتشاف الخاطئ لديه إلى (0.92) في بعض حالات التلقين. وعلى المستوى النوعي، أبدت النماذج قدرات مميزة في اكتشاف مخالفات التوثيق والأسلوب البرمجي التي لم يتمكن Pylint وFlake8 من رصدها، لكنها في المقابل فشلت في تحديد المخالفات الهيكلية المعقدة حتى في الحالات التي حققت فيها دقة شبه مثالية. كما تأثرت النتائج بأسلوب التلقين المستخدم، حيث كان التلقين البنيوي الأكثر توازنًا في معظم الحالات. وإضافةً إلى ذلك، بيّنت الدراسة أن استراتيجية إدخال البيانات تُعد متغيرًا مهمًا، إذ أدى التعامل مع مقاطع الكود بشكل فردي بدلًا من إدخالها على شكل دفعات إلى تحسن عام في الأداء، حيث تحقق أكبر تحسن في قيمة F1-Score بلغ زيادة (+0.64) لصالح ChatGPT على وجه الخصوص. وتقدم هذه الرسالة دليلًا تجريبيًا على فعالية استخدام نماذج اللغات الكبيرة في تحسين جودة الاكواد البرمجية، وتبرز إمكاناتها كأدوات تكميلية للتحليل الساكن لاكواد Python .

الكلمات المفتاحية:

نماذج اللغات الكبيرة, التحليل الساكن للكود, جودة الكود, مخالفات الكود, *بايثون.*